

FDU 数值算法 4. 快速 Fourier 变换

本文根据邵美悦老师课堂笔记整理而成，并参考以下教材：

- 数值逼近 (蒋尔雄 & 赵风光 & 苏仰峰, 第 2 版) 第 4, 6 章
- 数值分析 (李庆扬 & 王能超 & 易大义, 第 5 版) 第 3 章
- 数值计算方法 (吕同富 & 康兆敏 & 方秀男) 第 6 章

欢迎批评指正!

4.1 Fourier 分析

4.1.1 标准正交系

考虑 $[-\pi, \pi]$ 上的平方可积函数空间 $L^2([-\pi, \pi])$

其内积定义为 $\langle f, g \rangle_2 := \int_{-\pi}^{\pi} f(t) \overline{g(t)} dt$

其范数定义为 $\|f\|_2 := \langle f, f \rangle_2^{1/2} = \sqrt{\int_{-\pi}^{\pi} |f(t)|^2 dt}$

实数域上的一个标准正交系为 $\left\{ \frac{1}{\sqrt{2\pi}}, \frac{\cos(t)}{\sqrt{\pi}}, \frac{\sin(t)}{\sqrt{\pi}}, \frac{\cos(2t)}{\sqrt{\pi}}, \frac{\sin(2t)}{\sqrt{\pi}}, \dots \right\}$

复数域上的一个标准正交系为 $\left\{ \frac{1}{\sqrt{2\pi}}, \frac{\exp(-it)}{\sqrt{2\pi}}, \frac{\exp(it)}{\sqrt{2\pi}}, \frac{\exp(-2it)}{\sqrt{2\pi}}, \frac{\exp(2it)}{\sqrt{2\pi}}, \dots \right\}$

$$\begin{aligned} \langle \exp(imt), \exp(int) \rangle_2 &= \int_{-\pi}^{\pi} \exp(imt) \exp(-int) dt \\ &= \int_{-\pi}^{\pi} \exp(i(m-n)t) dt \\ &= \begin{cases} \int_{-\pi}^{\pi} 1 \cdot dt, & \text{if } m = n \\ \frac{1}{i(m-n)} \exp(i(m-n)t) \Big|_{-\pi}^{\pi}, & \text{if } m \neq n \end{cases} \\ &= \begin{cases} 2\pi, & \text{if } m = n \\ \frac{1}{i(m-n)} [\exp(i(m-n)\pi) - \exp(-i(m-n)\pi)], & \text{if } m \neq n \end{cases} \\ &= \begin{cases} 2\pi, & \text{if } m = n \\ 0, & \text{if } m \neq n \end{cases} \end{aligned}$$

一般地，考虑 $[-T/2, T/2]$ 上的平方可积函数空间 $L^2([-T/2, T/2])$

其内积定义为 $\langle f, g \rangle_2 := \int_{-T/2}^{T/2} f(t) \overline{g(t)} dt$

其范数定义为 $\|f\|_2 := \langle f, f \rangle_2^{1/2} = \sqrt{\int_{-T/2}^{T/2} |f(t)|^2 dt}$

实数域上的一个标准正交系为 $\left\{ \frac{1}{\sqrt{T}}, \frac{\cos(2\pi t/T)}{\sqrt{T/2}}, \frac{\sin(2\pi t/T)}{\sqrt{T/2}}, \frac{\cos(4\pi t/T)}{\sqrt{T/2}}, \frac{\sin(4\pi t/T)}{\sqrt{T/2}}, \dots \right\}$

复数域上的一个标准正交系为 $\left\{ \frac{1}{\sqrt{T}}, \frac{\exp(-2\pi it/T)}{\sqrt{T}}, \frac{\exp(i2\pi t/T)}{\sqrt{T}}, \frac{\exp(-4\pi it/T)}{\sqrt{T}}, \frac{\exp(4\pi it/T)}{\sqrt{T}}, \dots \right\}$

4.1.2 Fourier 级数

设 $f(t)$ 是定义在实数域上的函数。

当 $f(t)$ 是以 T 为周期的复值函数时，它可以展开为如下的实形式的 Fourier 级数：

$$\begin{aligned} f(t) &= \frac{a_0}{2} + \sum_{n=1}^{\infty} \left[a_n \cos\left(\frac{2n\pi t}{T}\right) + b_n \sin\left(\frac{2n\pi t}{T}\right) \right] \\ a_n &= \frac{2}{T} \left\langle f(t), \cos\left(\frac{2n\pi t}{T}\right) \right\rangle_2 = \frac{2}{T} \int_{-T/2}^{T/2} f(t) \cos\left(\frac{2n\pi t}{T}\right) dt \quad (n = 0, 1, 2, \dots) \\ b_n &= \frac{2}{T} \left\langle f(t), \sin\left(\frac{2n\pi t}{T}\right) \right\rangle_2 = \frac{2}{T} \int_{-T/2}^{T/2} f(t) \sin\left(\frac{2n\pi t}{T}\right) dt \quad (n = 1, 2, 3, \dots) \\ b_0 &= 0 \\ a_{-n} &= a_n \\ b_{-n} &= -b_n \end{aligned}$$

应用 Euler 恒等式 $e^{i\theta} = \cos(\theta) + i \sin(\theta)$ 可得复形式的 Fourier 级数：

$$f(t) = \sum_{n=-\infty}^{\infty} c_n \exp\left(\frac{2n\pi it}{T}\right)$$

$$\text{where } c_n = \frac{1}{2}(a_n - ib_n) = \frac{1}{T} \left\langle f(t), \exp\left(\frac{2n\pi it}{T}\right) \right\rangle_2 = \frac{1}{T} \int_{-T/2}^{T/2} f(t) \exp\left(-\frac{i2n\pi t}{T}\right) dt \quad (n = 0, \pm 1, \pm 2, \dots)$$

Here we use Cauchy principal value of an infinite sum, i.e. $\sum_{n=-\infty}^{\infty} := \lim_{N \rightarrow \infty} \sum_{n=-N}^N$

Note that $a_n = c_n + c_{-n}$, $b_n = i(c_n - c_{-n})$

显然复形式要比实形式干净很多.

当 f 为实值函数时, 我们有 $c_{-n} = \overline{c_n}$ 的共轭对称性, 此时 a_n 和 b_n 均为实数.

当 f 为复值函数时, 共轭对称性 $c_{-n} = \overline{c_n}$ 不一定成立, 此时 a_n 和 b_n 可能为复数.

从变换的角度讲, 由 $f(t)$ 可获得复数列 $\{c_n\}$, 同时根据 $\{c_n\}$ 又可还原函数 $f(t)$.

• (Riemann 引理)

函数的 Fourier 变换在无穷远处趋于零.

换言之, 其 Fourier 级数的系数趋于 0:

$$c_n = \frac{1}{T} \int_{-T/2}^{T/2} f(t) \exp\left(-\frac{i2n\pi t}{T}\right) dt \rightarrow 0 \quad (n \rightarrow \infty)$$

• (数值逼近, 例 6.1)

考虑以 2π 为周期的函数

$$f(t) := \begin{cases} 1, & 0 < t < \pi \\ 0, & -\pi < t < 0 \end{cases}$$

其 Fourier 系数为:

$$\begin{aligned} c_n &= \frac{1}{T} \left\langle f(t), \exp\left(\frac{2n\pi it}{T}\right) \right\rangle_2 \\ &= \frac{1}{T} \int_{-T/2}^{T/2} f(t) \exp\left(-\frac{i2n\pi t}{T}\right) dt \\ &= \frac{1}{2\pi} \int_{-\pi}^{\pi} f(t) \exp(-int) dt \\ &= \frac{1}{2\pi} \int_0^{\pi} \exp(-int) dt \\ &= \frac{1}{2\pi} \cdot \left(-\frac{1}{in}\right) \exp(-int) \Big|_0^{\pi} \\ &= \frac{i}{2n\pi} (\exp(-in\pi) - 1) \\ &\quad \left(\text{note that } \exp(in\pi) = \begin{cases} \exp(0) = 1, & \text{if } n \text{ is even} \\ \exp(i\pi) = -1, & \text{if } n \text{ is odd} \end{cases} \right) \\ &= \begin{cases} 0, & \text{if } n \text{ is even} \\ -\frac{i}{n\pi}, & \text{if } n \text{ is odd} \end{cases} \end{aligned}$$

因此 $f(t)$ 的 Fourier 级数为:

$$\begin{aligned} f(t) &= \sum_{n=-\infty}^{\infty} c_n \exp\left(\frac{i2n\pi t}{T}\right) \\ &= - \sum_{n=-\infty}^{\infty} \frac{i}{(2n+1)\pi} \exp(i(2n+1)t) \end{aligned}$$

(Fourier 级数的收敛性, 数值逼近, 定理 6.1)

考虑平方可积空间 $L^2([-T/2, T/2])$

其内积定义为 $\langle f, g \rangle_2 := \int_{-T/2}^{T/2} f(t) \overline{g(t)} dt$

其范数定义为 $\|f\|_2 := \langle f, f \rangle_2^{1/2} = \sqrt{\int_{-T/2}^{T/2} |f(t)|^2 dt}$

若周期为 T 的函数 $f(t)$ 在 $[-T/2, T/2]$ 上平方可积 (即 $\int_{-T/2}^{T/2} (f(t))^2 dt < \infty$),

则 $f(t)$ 的 Fourier 级数的部分和序列 $\{f_n\}$ 平方收敛到 $f(t)$:

$$\lim_{n \rightarrow \infty} \|f - f_n\|_2 = 0$$

$$\text{where } f_n(t) := \sum_{k=-n}^n c_k \exp\left(\frac{i2k\pi t}{T}\right)$$

$$\text{and } c_k = \frac{1}{T} \left\langle f(t), \exp\left(\frac{i2k\pi t}{T}\right) \right\rangle_2 = \frac{1}{T} \int_{-T/2}^{T/2} f(t) \exp\left(-\frac{i2k\pi t}{T}\right) dt$$

值得注意的是, $f_n(t)$ 恰好是 $f(t)$ 在子空间 $\text{span}\{1, \exp(\pm \frac{i2\pi t}{T}), \dots, \exp(\pm \frac{i2n\pi t}{T})\}$ 上的投影, 即 $f(t)$ 在子空间 $\text{span}\{1, \exp(\pm \frac{i2\pi t}{T}), \dots, \exp(\pm \frac{i2n\pi t}{T})\}$ 上的最佳平方三角逼近.

- Fourier series:

If $f(x)$ is smooth and periodic

(WLOG, we assume $f(x + 2\pi) = f(x)$), then

$$f(x) = \lim_{N \rightarrow \infty} \sum_{k=-N}^N c_k \exp(ikx),$$

where

$$c_k = \frac{1}{2\pi} \int_0^{2\pi} \exp(-ikx) f(x) dx$$

- Trapezoidal rule

$$c_k \approx \frac{1}{n} \sum_{j=0}^{n-1} \exp\left(-\frac{2jk\pi i}{n}\right) f\left(\frac{2j\pi}{n}\right)$$

4.1.3 Dirac 函数

Dirac 函数 $\delta(t) := \begin{cases} \infty & t = 0 \\ 0 & t \neq 0 \end{cases}$ 不像是一个严格定义的函数, 而更像是一个函数序列的极限,

例如 Gauss 函数序列 $\left\{g_n(t) := \frac{n}{\sqrt{2\pi}} \exp\left(-\frac{n^2 t^2}{2}\right)\right\}$

或 sinc 函数序列 $\left\{s_n(t) := n \cdot \text{sinc}(nt) = n \cdot \frac{\sin(n\pi t)}{n\pi t} = \frac{\sin(n\pi t)}{\pi t}\right\}$

在此意义下, δ 函数几乎处处为零, 但在点 $t = 0$ 处无限大.

该函数的无限高度和无限小的宽度相匹配, 使得其积分为 $\int \delta(t) dt = 1$.

- Dirac 函数最常用的定义为:

$$\begin{aligned} \delta(t) &:= \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp(i\omega t) d\omega \\ &= \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp(-i\omega t) d\omega \\ &= \int_{-\infty}^{\infty} \exp(i(2\pi\mu)t) d\mu \\ &= \int_{-\infty}^{\infty} \exp(-i(2\pi\mu)t) d\mu \end{aligned}$$

- (取样性质)

对于 Dirac 函数与任意连续函数 f 的卷积, 我们都有 $\int_a^b f(s) \delta(s - t) ds = f(t)$

其中 t 在积分区间 $[a, b]$ 中 (若不在, 则卷积结果为 0)

(邵老师补充示例)

从上述证明过程可以看出, 区间的大小其实并不重要.

本例实际上说明了:

$$\frac{1}{\pi} \lim_{t \rightarrow 0^+} \frac{t}{x^2 + t^2} = \delta(x)$$

下面是 Dirac 函数的常用逼近:

$$\begin{aligned}
\delta(x) &= \lim_{t \rightarrow 0+} \frac{1}{2\sqrt{\pi t}} \exp\left(-\frac{x^2}{4t}\right) \quad (\text{Gaussian}) \\
&= \frac{1}{\pi} \lim_{t \rightarrow 0+} \frac{1}{x} \sin\left(\frac{x}{t}\right) \quad (\text{sinc}) \\
&= \frac{1}{2} \lim_{t \rightarrow 0+} t|x|^{t-1} \quad (\text{rational}) \\
&= \frac{1}{2\pi} \lim_{n \rightarrow \infty} \frac{\sin\left(\frac{2n+1}{2}x\right)}{\sin\left(\frac{x}{2}\right)} \quad (\text{Dirichlet kernel})
\end{aligned}$$

4.1.4 Fourier 变换

(数值逼近, 定理 6.2)

若 $f(t)$ 是定义在整个实轴上的复值函数, 且绝对可积 (即 $\int_{-\infty}^{\infty} |f(t)| dt < \infty$), 则可定义其 **Fourier 变换**为:

$$\tilde{f}(\mu) := \int_{-\infty}^{\infty} f(t) \exp(-i2\pi\mu t) dt$$

上述条件只是 Fourier 变换存在的充分条件, 严格的条件比较复杂, 涉及广义函数.

若 $f(t)$ 是定义在整个实轴上的复值函数, 且满足 [Dirichlet 条件](#): (存疑)

- ① 周期性: 函数 f 是周期的, 即存在 $T > 0$ 使得 $f(t+T) = f(t)$ 恒成立.
- ② 有界性: 函数 f 是有界的, 即存在 $M > 0$ 使得 $|f(t)| \leq M$ 恒成立.
- ③ 绝对可积性: 函数 f 在周期长度的区间上是绝对可积的, 即 $\int_{-T/2}^{T/2} |f(t)| dt < \infty$ (与数值逼近定理 6.2 不太一样)
- ④ 有限个不连续点: 函数 f 只能有有限个不连续点, 并且在这些点处的跳跃幅度必须是有限的.
- ⑤ 有限个极值点: 函数 f 在任意区间内的极值点的个数必须是有限的.

则 Fourier 变换存在且可逆.

Fourier 变换 $\tilde{f}(\mu)$ 通常是复值函数, 我们记:

$$\begin{aligned}
\tilde{f}(\mu) &= \text{Re}(\tilde{f}(\mu)) + i \cdot \text{Im}(\tilde{f}(\mu)) \\
&= |\tilde{f}(\mu)| \exp(i \cdot \varphi(\mu)) \\
\text{where } \begin{cases} |\tilde{f}(\mu)| &:= \sqrt{[\text{Re}(\tilde{f}(\mu))]^2 + [\text{Im}(\tilde{f}(\mu))]^2} \\ \varphi(\mu) &:= \arctan\left(\frac{\text{Im}(\tilde{f}(\mu))}{\text{Re}(\tilde{f}(\mu))}\right) \end{cases}
\end{aligned}$$

其中我们称 $|\tilde{f}(\mu)|$ 为 **Fourier 频谱**, 称 $\varphi(\mu)$ 为**相位谱**.

(值得说明的是, $\arctan$ 应为四象限反正切函数, 例如 MATLAB 中的 `atan2(Imag, Real)` 函数)

功率谱的定义为 $|\tilde{f}(\mu)|^2 = [\text{Re}(\tilde{f}(\mu))]^2 + [\text{Im}(\tilde{f}(\mu))]^2$

可以证明**逆 Fourier 变换**如下:

$$\begin{aligned}
&\int_{-\infty}^{\infty} \tilde{f}(\mu) \exp(i2\pi\mu t) d\mu = f(t) \\
\hline
\int_{-\infty}^{\infty} \tilde{f}(\mu) \exp(i2\pi\mu t) d\mu &= \int_{-\infty}^{\infty} \left\{ \int_{-\infty}^{\infty} f(s) \exp(-i2\pi\mu s) ds \right\} \exp(i2\pi\mu t) d\mu \\
&= \int_{-\infty}^{\infty} f(s) \left\{ \int_{-\infty}^{\infty} \exp(-i2\pi(s-t)\mu) d\mu \right\} ds \\
&= \int_{-\infty}^{\infty} f(s) \delta(s-t) ds \quad (\text{use sampling property of Dirac delta function}) \\
&= f(t)
\end{aligned}$$

我们称 $f(t) \leftrightarrow \tilde{f}(\mu)$ 为一个 **Fourier 变换对** (详细的表格可参考 [Fourier Table](#))

工程上, 我们通常将 $f(t)$ 看成关于时间 t 的函数, 对应于**空间域**;

将 Fourier 变换 $\tilde{f}(\mu)$ 看成关于频率 μ 的函数, 对应于**频率域**.

Fourier 变换可以推广至高维形式, 以二维为例:

$$\begin{aligned}\tilde{f}(\mu, \nu) &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(t, s) \exp(-i2\pi(\mu t + \nu s)) dt ds \\ f(t, s) &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \tilde{f}(\mu, \nu) \exp(i2\pi(\mu t + \nu s)) d\mu d\nu\end{aligned}$$

二维 Fourier 变换具有可分离性，即可转化为逐次一维 Fourier 变换。

例如先做一次关于 t 的一维 Fourier 变换，再做一次关于 s 的一维 Fourier 变换：

$$\begin{aligned}\tilde{f}(\mu, \nu) &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(t, s) \exp(-i2\pi(\mu t + \nu s)) dt ds \\ &= \int_{-\infty}^{\infty} \left\{ \int_{-\infty}^{\infty} f(t, s) \exp(-i2\pi\mu t) dt \right\} \exp(-i2\pi\nu s) ds\end{aligned}$$

二维逆 Fourier 变换也是一样：

$$\begin{aligned}f(t, s) &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \tilde{f}(\mu, \nu) \exp(i2\pi(\mu t + \nu s)) d\mu d\nu \\ &= \int_{-\infty}^{\infty} \left\{ \int_{-\infty}^{\infty} \tilde{f}(\mu, \nu) \exp(i2\pi\mu s) d\mu \right\} \exp(i2\pi\nu s) d\nu\end{aligned}$$

事实上，高维 Fourier 变换都可通过逐次一维 Fourier 变换实现。

因此我们只需要深入研究一维 Fourier 变换的性质和处理技巧就足够了。

- Fourier transform:

$$F(s) = \int_{-\infty}^{+\infty} e^{-ist} f(t) dt$$

- Inverse Fourier transform:

$$f(t) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} e^{ist} F(s) ds$$

- Equispaced discretization leads to discrete Fourier transform

4.1.5 卷积定理

关于离散变量的两个函数的卷积是指将一个函数关于其原点翻转 (旋转 180°)，并将其滑过另一个函数。

在滑动过程中，对每个位置执行乘积求和运算。

离散卷积可以用来做多项式乘法：

$$\begin{cases} x = [1, 1] \\ y = [1, 2, 1] \end{cases} \Rightarrow \text{conv}(x, y) = [1, 3, 3, 1]$$

进而可以用来做大数乘法，计算复杂度为 $O(n \log(n))$ (其中 n 是参与运算的大数的位数)

普通的离散卷积可以通过适当的零填充得到循环卷积：

$$\begin{cases} \tilde{x} = [1, 1, 0, 0, 0, 0] \\ \tilde{y} = [1, 2, 1, 0, 0, 0] \end{cases} \Rightarrow \text{circular_conv}(x, y) = \text{conv}(\tilde{x}, \tilde{y}) = [1, 3, 3, 1, 0, 0]$$

类似地，关于连续变量 t 的两个连续函数 f, g 的卷积 $f \star g$ 定义为：

$$(f \star g)(t) := \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$

其中 τ 是积分虚拟变量，负号代表翻转， t 是函数 g 滑过函数 f 的位移。

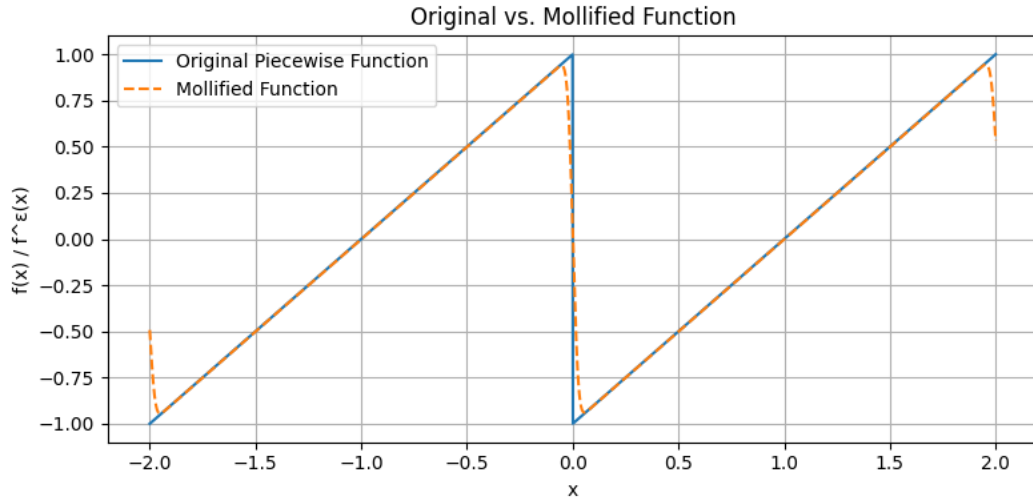
卷积运算满足：

- ① 交换律: $f \star g = g \star f$
- ② 结合律: $f \star (g \star h) = (f \star g) \star h$
- ③ 分配律: $f \star (g + h) = (f \star g) + (f \star h)$

卷积运算的单位元就是 Dirac 函数:

$$\begin{aligned} f(t) &= \int_{-\infty}^{\infty} \delta(s) f(t-s) ds \\ &= \int_{-\infty}^{\infty} f(s) \delta(t-s) ds \end{aligned}$$

此外，通过与磨光函数 (例如 Gauss 函数) 进行卷积可以将非连续函数磨光，即变为无限光滑的函数:



Fourier 变换在处理卷积时非常有用，因为**卷积的 Fourier 变换是被卷积的两个函数的 Fourier 变换的乘积**:
考虑卷积 $h = f \star g$:

$$\begin{aligned} \tilde{h}(\mu) &= \int_{-\infty}^{\infty} h(t) \exp(-i2\pi\mu t) dt \\ &= \int_{-\infty}^{\infty} \left\{ \int_{-\infty}^{\infty} f(s) g(t-s) ds \right\} \exp(-i2\pi\mu t) dt \\ &= \int_{-\infty}^{\infty} f(s) \exp(-i2\pi\mu s) \left\{ \int_{-\infty}^{\infty} g(t-s) \exp(-i2\pi\mu(t-s)) dt \right\} ds \\ &= \int_{-\infty}^{\infty} f(s) \exp(-i2\pi\mu s) \left\{ \int_{-\infty}^{\infty} g(\tau) \exp(-i2\pi\mu\tau) d\tau \right\} ds \quad (\text{denote } \tau := t-s) \\ &= \int_{-\infty}^{\infty} f(s) \exp(-i2\pi\mu s) ds \cdot \int_{-\infty}^{\infty} g(\tau) \exp(-i2\pi\mu\tau) d\tau \\ &= \tilde{f}(\mu) \cdot \tilde{g}(\mu) \end{aligned}$$

因此空间域的卷积对应于频率域的乘积.

从中我们还可以推出 **Parseval 定理**: (本质上是因为 $\|\cdot\|_2$ 是酉不变范数)

$$\|f\|_2 = \int_{-\infty}^{\infty} |f(t)|^2 dt = \int_{-\infty}^{\infty} |\tilde{f}(\mu)|^2 d\mu = \|\tilde{f}\|_2$$

反过来，空间域的乘积对应于频率域的卷积.

考虑乘积 $h = f \cdot g$:

$$\begin{aligned} \tilde{h}(\mu) &= \int_{-\infty}^{\infty} h(t) \exp(-i2\pi\mu t) dt \\ &= \int_{-\infty}^{\infty} f(t) g(t) \exp(-i2\pi\mu t) dt \\ &= \int_{-\infty}^{\infty} \left\{ \int_{-\infty}^{\infty} \tilde{f}(\nu) \exp(i2\pi\nu t) d\nu \right\} g(t) \exp(-i2\pi\mu t) dt \\ &= \int_{-\infty}^{\infty} \tilde{f}(\nu) \left\{ \int_{-\infty}^{\infty} g(t) \exp(-i2\pi(\mu-\nu)t) dt \right\} d\nu \\ &= \int_{-\infty}^{\infty} \tilde{f}(\nu) \tilde{g}(\mu-\nu) d\nu \\ &= \tilde{f}(\mu) \star \tilde{g}(\mu) \end{aligned}$$

设 $\{f[k]\}$ 和 $\{g[k]\}$ 是以 N 为周期的点列，则其 (循环) 离散卷积定义为:

$$(f \star g)[k] := \sum_{j=0}^{N-1} f[j] \cdot g[k-j] \quad (k = 0, 1, \dots, N-1)$$

类似于连续卷积，离散卷积也满足卷积定理。

即**卷积的离散 Fourier 变换是被卷积的两个点列的离散 Fourier 变换的乘积**：

考虑卷积 $h = f \star g$ ：

$$\begin{aligned} \tilde{h}_{\text{DFT}}[j] &= \sum_{k=0}^{N-1} h[k] \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) \\ &= \sum_{k=0}^{N-1} \left\{ \sum_{l=0}^{N-1} f[l] \cdot g[k-l] \right\} \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) \\ &= \sum_{l=0}^{N-1} f[l] \exp\left(-\frac{\mathrm{i}2\pi jl}{N}\right) \left\{ \sum_{k=0}^{N-1} g[k-l] \exp\left(-\frac{\mathrm{i}2\pi j(k-l)}{N}\right) \right\} \\ &= \sum_{l=0}^{N-1} f[l] \exp\left(-\frac{\mathrm{i}2\pi jl}{N}\right) \cdot \sum_{k=0}^{N-1} g[k] \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) \\ &= \tilde{f}_{\text{DFT}}[j] \cdot \tilde{g}_{\text{DFT}}[j] \end{aligned}$$

因此空间域的卷积对应于频率域的乘积。

从中我们还可以推出 **Parseval 定理**：

$$\sum_{k=0}^{N-1} |f[k]|^2 = \frac{1}{N} \sum_{j=0}^{N-1} |\tilde{f}_{\text{DFT}}[j]|^2$$

反过来，空间域的乘积对应于频率域的卷积。

考虑乘积 $h = f \cdot g$ ：

$$\begin{aligned} \tilde{h}_{\text{DFT}}[j] &= \sum_{k=0}^{N-1} h[k] \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) \\ &= \sum_{k=0}^{N-1} f[k]g[k] \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) \\ &= \sum_{k=0}^{N-1} \left\{ \frac{1}{N} \sum_{l=0}^{N-1} \tilde{f}_{\text{DFT}}[l] \exp\left(\frac{\mathrm{i}2\pi lk}{N}\right) \right\} g[k] \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) \\ &= \frac{1}{N} \sum_{l=0}^{N-1} \tilde{f}_{\text{DFT}}[l] \left\{ \sum_{k=0}^{N-1} g[k] \exp\left(-\frac{\mathrm{i}2\pi(j-l)k}{N}\right) \right\} \\ &= \frac{1}{N} \sum_{l=0}^{N-1} \tilde{f}_{\text{DFT}}[l] \cdot \tilde{g}_{\text{DFT}}[j-l] \\ &= \frac{1}{N} (\tilde{f}_{\text{DFT}} \star \tilde{g}_{\text{DFT}})[j] \end{aligned}$$

4.2 离散 Fourier 变换

尽管 Fourier 级数和 Fourier 变换都具有非常优美的数学结构，但使用它们在有限字长的计算机上进行数值计算是不切实际的。为此，我们必须建立 Fourier 变换的数值方法。

4.2.1 三角插值

设 $f(t)$ 是定义在实数域上的周期为 T 的复值函数。

给定 $[0, T]$ 上的 $2N + 1$ 个等距节点：

$$\begin{aligned} t_j &= jh \quad (j = 0, 1, \dots, 2N) \\ \text{where } h &:= \frac{T}{2N + 1} \end{aligned}$$

假设三角插值问题的解具有如下形式：

$$\begin{aligned}
T_N(t) &= \frac{\tilde{a}_0}{2} + \sum_{k=1}^N \left[\tilde{a}_k \cos\left(\frac{2k\pi t}{T}\right) + \tilde{b}_k \sin\left(\frac{2k\pi t}{T}\right) \right] \\
&= \sum_{k=-N}^N \tilde{c}_k \exp\left(\frac{i2k\pi t}{T}\right)
\end{aligned}$$

根据插值条件 $T_N(t_j) = f(t_j)$ ($j = 0, 1, \dots, 2N$) 得到的 Vandermonde 系统如下:

$$\begin{aligned}
\vec{f} = \begin{bmatrix} f(t_0) \\ f(t_1) \\ \vdots \\ f(t_{2N}) \end{bmatrix} &= \begin{bmatrix} T_N(t_0) \\ T_N(t_1) \\ \vdots \\ T_N(t_{2N}) \end{bmatrix} = \begin{bmatrix} \omega^0 & \omega^0 & \cdots & \omega^0 \\ \omega^{-N} & \omega^{-N+1} & \cdots & \omega^N \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{2N \cdot (-N)} & \omega^{2N \cdot (-N+1)} & \cdots & \omega^{2N \cdot N} \end{bmatrix} \begin{bmatrix} \tilde{c}_{-N} \\ \tilde{c}_{-N+1} \\ \vdots \\ \tilde{c}_N \end{bmatrix} = V \cdot \vec{c} \\
&\text{where } \omega := \exp\left(\frac{i2\pi}{2N+1}\right) \\
&\text{and } V_{j,k} = \exp\left(\frac{i2k\pi \cdot t_j}{T}\right) = \exp\left(\frac{i2k\pi}{T} \cdot \frac{jT}{2N+1}\right) = \exp\left(jk \cdot \frac{i2\pi}{2N+1}\right) = \omega^{jk}
\end{aligned}$$

由于 $\omega^{-N}, \omega^{-N+1}, \dots, \omega^N$ 互不相同, 故 Vandermonde 矩阵 $V \in \mathbb{C}^{(2N+1) \times (2N+1)}$ 非奇异. 因此三角插值问题的解存在且唯一, 其系数为:

$$\begin{aligned}
\tilde{a}_k &= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot \operatorname{Re}(\omega^{-jk}) \quad (\text{note that } \operatorname{Re}(\omega^{-jk}) = \operatorname{Re}(\omega^{jk})) \\
&= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot \operatorname{Re}(\omega^{jk}) \\
&= \frac{2}{2N+1} \sum_{j=0}^{2N} f\left(\frac{jT}{2N+1}\right) \cos\left(\frac{2jk\pi}{2N+1}\right) \quad (k = 0, 1, \dots, N) \\
\tilde{b}_k &= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot [-\operatorname{Im}(\omega^{-jk})] \quad (\text{note that } \operatorname{Im}(\omega^{-jk}) = -\operatorname{Im}(\omega^{jk})) \\
&= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot \operatorname{Im}(\omega^{jk}) \\
&= \frac{2}{2N+1} \sum_{j=0}^{2N} f\left(\frac{jT}{2N+1}\right) \sin\left(\frac{2jk\pi}{2N+1}\right) \quad (k = 1, 2, \dots, N) \\
\hline
\tilde{c}_k &= \frac{2}{2N+1} \sum_{j=0}^{2N} f(t_j) \cdot \omega^{-jk} \\
&= \frac{1}{2N+1} \sum_{j=0}^{2N} f\left(\frac{jT}{2N+1}\right) \exp\left(-\frac{i2jk\pi}{2N+1}\right) \quad (k = 0, \pm 1, \dots, \pm N)
\end{aligned}$$

有趣的是, $\tilde{a}_k, \tilde{b}_k, \tilde{c}_k$ 可通过对 a_k, b_k, c_k 运用梯形求积公式做数值积分得到, 节点为 $t_j = \frac{jT}{2N+1}$ ($j = 0, 1, \dots, 2N+1$), 其中 a_k, b_k, c_k 为:

$$\begin{aligned}
a_k &= \frac{2}{T} \int_{-T/2}^{T/2} f(t) \cos\left(\frac{2k\pi t}{T}\right) dt \\
b_k &= \frac{2}{T} \int_{-T/2}^{T/2} f(t) \sin\left(\frac{2k\pi t}{T}\right) dt \\
c_k &= \frac{1}{T} \int_{-T/2}^{T/2} f(t) \exp\left(-\frac{i2k\pi t}{T}\right) dt
\end{aligned}$$

- Least squares approximation in $\text{span}\{\exp(-iNx), \dots, \exp(iNx)\}$: truncated Fourier series

$$p(x) = \frac{1}{2\pi} \sum_{k=-N}^N \exp(ikx) \int_0^{2\pi} \exp(-ikt) f(t) dt$$

- Using trapezoidal rule to approximate Fourier coefficients

$$\tilde{p}(x) = \frac{1}{2N+1} \sum_{k=-N}^N \sum_{j=0}^{2N} \exp\left(ikx - \frac{2jk\pi i}{2N+1}\right) f\left(\frac{2j\pi}{2N+1}\right)$$

It can be verified that $\tilde{p}\left(\frac{2j\pi}{2N+1}\right) = f\left(\frac{2j\pi}{2N+1}\right)$ for each j
 $\rightsquigarrow$ trigonometric interpolation with equispaced nodes

4.2.2 Fourier 积分的离散化

由于连续 Fourier 变换无法用计算机进行逻辑运算，
 因此在实际应用中，我们通常采用抽样的办法，观测 $f(t)$ 的一些离散值，
 然后利用数值积分将 Fourier 变换离散化。

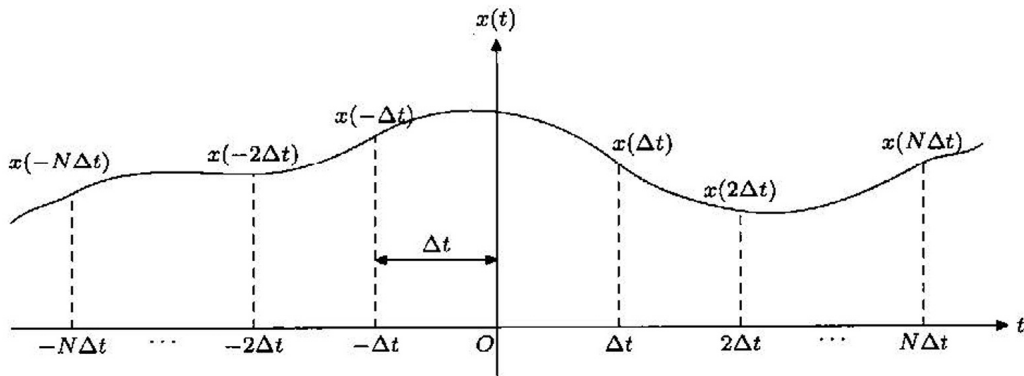


图 6.1

函数抽样是函数插值的逆过程。

设抽样得到的点列为 $f(-N\Delta t), \dots, f(-\Delta t), f[0], f(\Delta t), \dots, f(N\Delta t)$

当 N 充分大时，应用复化梯形公式，我们有：

$$\begin{aligned} \tilde{f}(\mu) &= \int_{-\infty}^{\infty} f(t) \exp(-i2\pi\mu t) dt \\ &\approx \int_{-N\Delta t}^{N\Delta t} f(t) \exp(-i2\pi\mu t) dt \\ &\approx \Delta t \sum_{j=-N}^N f(j\Delta t) \exp(-i2\pi\mu \cdot j\Delta t) \end{aligned}$$

取 $\mu = \mu_k := \frac{k}{N\Delta t}$ ($k = 0, 1, \dots, N-1$) 可得：

$$\begin{aligned} \tilde{f}\left(\frac{k}{N\Delta t}\right) &\approx \Delta t \sum_{j=-N}^N f(j\Delta t) \exp\left(-i2\pi \cdot \frac{k}{N\Delta t} \cdot j\Delta t\right) \\ &= \Delta t \sum_{j=-N}^N f(j\Delta t) \exp\left(-\frac{i2\pi jk}{N}\right) \end{aligned}$$

这就是一维 Fourier 变换的离散化。

4.2.3 离散 Fourier 变换

更一般地，给定复数列 $f[0], f[1], \dots, f[N-1]$

我们定义其**离散 Fourier 变换** (Discrete Fourier Transformation, DFT):

$$\tilde{f}_{\text{DFT}}[k] := \sum_{j=0}^{N-1} f[j] \exp\left(-\frac{i2\pi jk}{N}\right) \quad (k = 0, 1, \dots, N-1)$$

上述序列可通过以下变换恢复原序列:

$$\begin{aligned} \sum_{k=0}^{N-1} \tilde{f}_{\text{DFT}}[k] \cdot \exp\left(\frac{i2\pi jk}{N}\right) &= \sum_{k=0}^{N-1} \left\{ \sum_{l=0}^{N-1} f[l] \exp\left(-\frac{i2\pi lk}{N}\right) \right\} \cdot \exp\left(\frac{i2\pi jk}{N}\right) \\ &= \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} f[l] \exp\left(\frac{i2\pi(j-l)k}{N}\right) \\ &= \sum_{l=0}^{N-1} f[l] \sum_{k=0}^{N-1} \left[\exp\left(\frac{i2\pi(j-l)k}{N}\right) \right]^k \\ &\quad \left(\text{note that } \sum_{k=0}^{N-1} \left[\exp\left(\frac{i2\pi(j-l)k}{N}\right) \right]^k = \begin{cases} N, & \text{if } l = j \\ \frac{1 - \exp(i2\pi(j-l))}{1 - \exp(i2\pi(j-l)/N)}, & \text{if } l \neq j \end{cases} \right) \\ &= N \cdot f[j] + \sum_{\substack{l=0 \\ l \neq j}}^{N-1} f[l] \cdot \frac{1 - \exp(i2\pi(j-l))}{1 - \exp(i2\pi(j-l)/N)} \\ &\quad (\text{note that } \exp(i2\pi(j-l)) = 1 \text{ and } \exp(i2\pi(j-l)/N) \neq 1 \text{ if } l \neq j) \\ &= N \cdot f[j] + 0 \\ &= N \cdot f[j] \end{aligned}$$

于是我们有以下**离散 Fourier 逆变换**:

$$f[j] = \frac{1}{N} \sum_{k=0}^{N-1} \tilde{f}_{\text{DFT}}[k] \cdot \exp\left(\frac{i2\pi jk}{N}\right) \quad (j = 0, 1, \dots, N-1)$$

我们称 $\{f[j]\}_{j=0}^{N-1}$ 和 $\{\tilde{f}_{\text{DFT}}[k]\}_{k=0}^{N-1}$ 为**离散 Fourier 变换对**，记为 $f[j] \Leftrightarrow \tilde{f}_{\text{DFT}}[k]$

离散 Fourier 变换的这种可恢复性质在工程技术中有着极为重要的应用，

我们可以通过抽样获得一组离散值，并利用离散 Fourier 变换转化为另一组数据，

通过对变换后的数据的修改以及离散 Fourier 逆变换，达到对原数据的修正。

应用离散 Fourier 变换做数值分析，抽样也变得相当简单。

我们可以抽取函数的任一片段，而无须像在 Fourier 变换离散化那样做对称抽样：

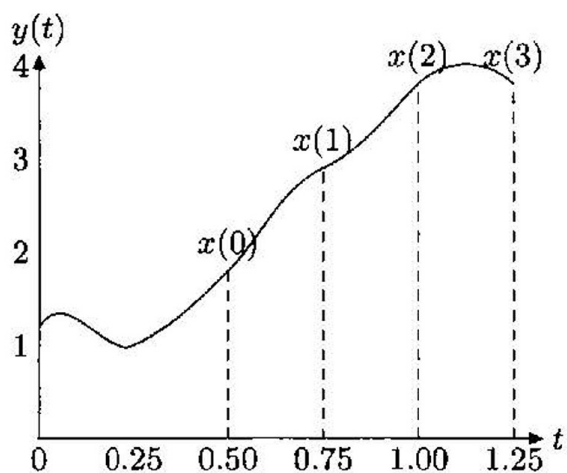


图 6.2

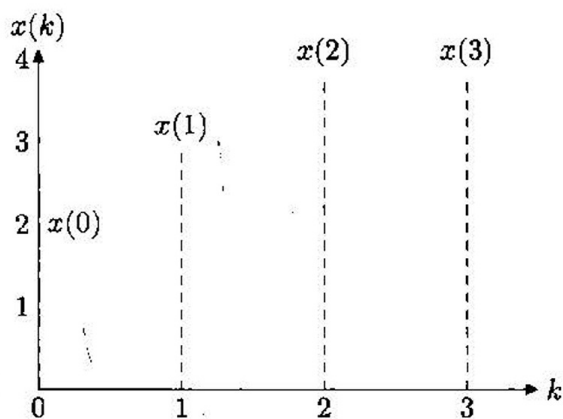


图 6.3

例3 作为实例, 考虑如图 6.2 所示的函数 $y(t)$ 应用 DFT 做抽样转换.

解 如果将此函数在自变量值为 0.5, 0.75, 1.0, 1.25 处抽样, 令 $x(k) = y(0.5 + 0.25k)$, $k = 0, 1, 2, 3$. 得到图 6.3 所示的离散抽样函数, 应用这 4 个抽样值得到如下的变换序列:

$$\begin{aligned}
 X(0) &= \sum_{k=0}^3 x(k)e^{j0} = x(0) + x(1) + x(2) + x(3) = 13, \\
 X(1) &= \sum_{k=0}^3 x(k)e^{-j2\pi k/4} = 2e^0 + 3e^{-j\pi/2} + 4e^{-j\pi} + 4e^{-j3\pi/2} = -2 + j, \\
 X(2) &= \sum_{k=0}^3 x(k)e^{-j4\pi k/4} = 2e^0 + 3e^{-j\pi} + 4e^{-j2\pi} + 4e^{-j3\pi} = -1, \\
 X(3) &= \sum_{k=0}^3 x(k)e^{-j6\pi k/4} = 2e^0 + 3e^{-j3\pi/2} + 4e^{-j3\pi} + 4e^{-j9\pi/2} = -2 - j.
 \end{aligned}$$

(一个有趣的观点)

N 个点的离散 Fourier 变换在时间维度和频率维度的周期均为 N :

$$\begin{aligned}
 \tilde{f}_{\text{DFT}}[k] &= \sum_{j=0}^{N-1} f[j] \exp\left(-\frac{j2\pi jk}{N}\right) \\
 &= \sum_{j=0}^{N-1} f[j] \exp\left(-\frac{j2\pi(j \pm sN)(k \pm pN)}{N}\right) \quad (\text{for any integer } s, p)
 \end{aligned}$$

设物理采样间隔为 ΔT , 则 DFT 的频率分辨率为 $1/(N\Delta T)$,

第 k 个频率分量为 $k/(N\Delta T)$ ($k = 0, 1, \dots, N-1$),

最大有效物理频率为 $\lfloor N/2 \rfloor / (N\Delta T) = 1/(2\Delta T)$, 有效物理频率范围为 $[0, \frac{1}{2\Delta T}]$

带限函数是一类仅在以原点为中心的有限区间 $[-\mu_{\max}, \mu_{\max}]$ 内 Fourier 变换值非零的函数.

Nyquist 采样定理:

若以超过 2 倍带限 $2\mu_{\max}$ 的频率采样, 则连续带限函数就能够完全由其样本集合复原.

- Discrete Fourier transform (DFT):

Given $x_0, x_1, \dots, x_{n-1} \in \mathbb{C}$

Compute

$$X_k = \sum_{j=0}^{n-1} \exp\left(-\frac{2\pi i}{n} \cdot jk\right) x_j$$

for $k = 0, 1, \dots, n-1$

- Motivation

- Fourier series
- Trigonometric interpolation
- (Continuous) Fourier transform

- Matrix form: $X = F_n x$, e.g.,

$$\begin{bmatrix} X_0 \\ X_1 \\ X_2 \\ X_3 \end{bmatrix} = \begin{bmatrix} \omega_4^0 & \omega_4^0 & \omega_4^0 & \omega_4^0 \\ \omega_4^0 & \omega_4^1 & \omega_4^2 & \omega_4^3 \\ \omega_4^0 & \omega_4^2 & \omega_4^4 & \omega_4^6 \\ \omega_4^0 & \omega_4^3 & \omega_4^6 & \omega_4^9 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix},$$

where $\omega_n = \exp(-2\pi i/n)$

- Property: $F_n^* F_n = \bar{F}_n F_n = nI_n$
- Inverse discrete Fourier transform: $x = \bar{F}_n X/n$
 $\rightsquigarrow$ nearly identical to DFT
- Naive implementation: $O(n^2)$
- Fast Fourier transform (Gauss/Cooley–Tukey): $\Theta(n \log n)$

不同于连续情形，离散 Fourier 变换对总是存在的，并且具有与连续 Fourier 变换类似的性质。

例如高维离散 Fourier 变换可以通过逐次一维离散 Fourier 变换得到，即具有可分离性。

此外，线性可加、时间平移、频率平移、尺度性质等均适用于离散 Fourier 变换。

二维离散 Fourier 变换如下：

$$\tilde{f}_{\text{DFT}}[n, m] = \sum_{j=0}^{N-1} \sum_{k=0}^{M-1} f[j, k] \exp\left(-i2\pi \left(\frac{jn}{N} + \frac{km}{M}\right)\right)$$

where $n = 0, 1, \dots, N-1$ and $m = 0, 1, \dots, M-1$

$\Updownarrow$

$$f[j, k] = \frac{1}{NM} \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \tilde{f}_{\text{DFT}}[n, m] \exp\left(i2\pi \left(\frac{jn}{N} + \frac{km}{M}\right)\right)$$

where $j = 0, 1, \dots, N-1$ and $k = 0, 1, \dots, M-1$

二维离散 Fourier 变换可以通过逐次一维离散 Fourier 变换得到：

$$\begin{aligned} \tilde{f}_{\text{DFT}}[n, m] &= \sum_{j=0}^{N-1} \sum_{k=0}^{M-1} f[j, k] \exp\left(-i2\pi \left(\frac{jn}{N} + \frac{km}{M}\right)\right) \\ &= \sum_{j=0}^{N-1} \left\{ \sum_{k=0}^{M-1} f[j, k] \exp\left(-i2\pi \frac{km}{M}\right) \right\} \exp\left(-i2\pi \frac{jn}{N}\right) \\ &= \sum_{k=0}^{M-1} \left\{ \sum_{j=0}^{N-1} f[j, k] \exp\left(-i2\pi \frac{jn}{N}\right) \right\} \exp\left(-i2\pi \frac{km}{M}\right) \end{aligned}$$

(存疑: 另一种观点)

我们可以将二维网格 $\{f[j, k]\}$ 按行拉伸为列向量, 换言之, 将索引 $[j, k]$ 映射为 $jM + k$.

同时将二维网格 $\{\tilde{f}_{\text{DFT}}[n, m]\}$ 按列拉伸为列向量, 换言之, 将索引 $[n, m]$ 映射为 $n + mN$.

以 $N = 2, M = 3$ 为例:

$$f : \begin{bmatrix} (0,0) & (0,1) & (0,2) \\ (1,0) & (1,1) & (1,2) \end{bmatrix} \Rightarrow \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix}$$

$$\tilde{f}_{\text{DFT}} : \begin{bmatrix} (0,0) & (0,1) & (0,2) \\ (1,0) & (1,1) & (1,2) \end{bmatrix} \Rightarrow \begin{bmatrix} 0 & 2 & 4 \\ 1 & 3 & 5 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix}$$

$$\begin{aligned} \tilde{f}_{\text{DFT}}[n, m] &= \sum_{j=0}^{N-1} \sum_{k=0}^{M-1} f[j, k] \exp\left(-i2\pi\left(\frac{jn}{N} + \frac{km}{M}\right)\right) \\ &= \sum_{j=0}^{N-1} \sum_{k=0}^{M-1} f[j, k] \exp\left(-\frac{i2\pi(jnM + kmN)}{NM}\right) \end{aligned}$$

- 2-D discrete Fourier transform:

Given $(\mathbf{x}_{j_1, j_2}) \in \mathbb{C}^{n_1 \times n_2}$ for $0 \leq j_1 \leq n_1 - 1, 0 \leq j_2 \leq n_2 - 1$

Compute

$$X_{k_1, k_2} = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} \exp\left(-\frac{2j_1 k_1 \pi i}{n_1} - \frac{2j_2 k_2 \pi i}{n_2}\right) x_{j_1, j_2}$$

for $0 \leq k_1 \leq n_1 - 1, 0 \leq k_2 \leq n_2 - 1$

$\rightsquigarrow$ 1-D FFTs in each dimension

- Inverse Fourier transform

$$x_{j_1, j_2} = \frac{1}{n_1 n_2} \sum_{k_1=0}^{n_1-1} \sum_{k_2=0}^{n_2-1} \exp\left(\frac{2j_1 k_1 \pi i}{n_1} + \frac{2j_2 k_2 \pi i}{n_2}\right) X_{k_1, k_2}$$

- Complexity: $\Theta(N \log N)$, $N = n_1 n_2$

4.2.4 矩阵形式

考虑一维离散 Fourier 变换:

$$\begin{aligned} \tilde{f}_{\text{DFT}}[j] &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{i2\pi jk}{N}\right) \quad (j = 0, 1, \dots, N-1) \\ &\quad \Updownarrow \\ f[k] &= \frac{1}{N} \sum_{j=0}^{N-1} \tilde{f}_{\text{DFT}}[j] \cdot \exp\left(\frac{i2\pi jk}{N}\right) \quad (k = 0, 1, \dots, N-1) \end{aligned}$$

一维离散 Fourier 变换可表示为以下矩阵运算:

$$\vec{f}_{\text{DFT}} = \begin{bmatrix} \tilde{f}_{\text{DFT}}[0] \\ \tilde{f}_{\text{DFT}}[1] \\ \vdots \\ \tilde{f}_{\text{DFT}}[N-1] \end{bmatrix} = \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \dots & \omega^{0 \cdot (N-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \dots & \omega^{1 \cdot (N-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(N-1) \cdot 0} & \omega^{(N-1) \cdot 1} & \dots & \omega^{(N-1) \cdot (N-1)} \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ \vdots \\ f[N-1] \end{bmatrix} = V \cdot \vec{f}$$

where $\omega := \exp\left(-\frac{\mathrm{i}2\pi}{N}\right)$

and $V_{j,k} = \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) = \omega^{jk}$

由于 $\omega^0, \omega^1, \dots, \omega^{N-1}$ 互不相同, 故 Vandermonde 矩阵 $V \in \mathbb{C}^{N \times N}$ 非奇异.
可以证明 $V^H V = \bar{V} V = N \cdot I_N$:

$$\begin{aligned} (\bar{V} V)_{[j,k]} &= \sum_{l=0}^{N-1} \bar{V}_{j,l} \cdot V_{l,k} \\ &= \sum_{l=0}^{N-1} \omega^{-jl} \cdot \omega^{lk} \\ &= \sum_{l=0}^{N-1} \left[\omega^{(k-l)} \right]^l \\ &= \begin{cases} N, & \text{if } k = l \\ \frac{1 - \omega^{(k-l)N}}{1 - \omega^{(k-l)}} = \frac{0}{1 - \omega^{(k-l)}} = 0, & \text{if } k \neq l \end{cases} \\ &= N \cdot \delta_{j,k} \end{aligned}$$

这说明 Vandermonde 矩阵 V 与酉矩阵只差了常数倍, 因此问题是良态的.
同时我们知道一维离散 Fourier 逆变换可表示为以下矩阵运算:

$$\begin{aligned} \vec{f} &= V^{-1} \vec{f}_{\text{DFT}} = \frac{1}{N} V^H \vec{f}_{\text{DFT}} = \frac{1}{N} \bar{V} \vec{f}_{\text{DFT}} \\ &\quad \Updownarrow \\ \vec{f} &= \begin{bmatrix} f[0] \\ f[1] \\ \vdots \\ f[N-1] \end{bmatrix} = \frac{1}{N} \begin{bmatrix} \omega^{-0 \cdot 0} & \omega^{-0 \cdot 1} & \dots & \omega^{-0 \cdot (N-1)} \\ \omega^{-1 \cdot 0} & \omega^{-1 \cdot 1} & \dots & \omega^{-1 \cdot (N-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{-(N-1) \cdot 0} & \omega^{-(N-1) \cdot 1} & \dots & \omega^{-(N-1) \cdot (N-1)} \end{bmatrix} \begin{bmatrix} \tilde{f}_{\text{DFT}}[0] \\ \tilde{f}_{\text{DFT}}[1] \\ \vdots \\ \tilde{f}_{\text{DFT}}[N-1] \end{bmatrix} = \frac{1}{N} \bar{V} \cdot \vec{f}_{\text{DFT}} \\ &\quad \text{where } \omega := \exp\left(-\frac{\mathrm{i}2\pi}{N}\right) \\ &\quad \text{and } \bar{V}_{j,k} = \exp\left(\frac{\mathrm{i}2\pi jk}{N}\right) = \omega^{-jk} \end{aligned}$$

4.3 快速 Fourier 变换

考虑一维离散 Fourier 变换:

$$\begin{aligned} \tilde{f}_{\text{DFT}}[j] &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{\mathrm{i}2\pi jk}{N}\right) \quad (j = 0, 1, \dots, N-1) \\ &\quad \Updownarrow \\ f[k] &= \frac{1}{N} \sum_{j=0}^{N-1} \tilde{f}_{\text{DFT}}[j] \cdot \exp\left(\frac{\mathrm{i}2\pi jk}{N}\right) \quad (k = 0, 1, \dots, N-1) \end{aligned}$$

如果我们不考虑复指数部分的运算,
则求解离散 Fourier 变换共需 N^2 次复数乘法和 $N(N-1)$ 次复数加法.

当 N 较大时, 其计算量是很大的.

为此, Cooley 和 Tukey 提出了一种专门用于计算 DFT 的快速算法 (Fast Fourier Transformation, FFT),
它可将离散 Fourier 变换的计算复杂度从 $\Theta(N^2)$ 降低到 $\Theta(N \log(N))$,
被列为 20 世纪十大算法之一:

- 1946. Monte Carlo method
- 1947. Simplex method for linear programming
- 1950. Krylov subspace iteration method
- 1951. Decompositional approach to matrix computations
- 1957. Fortran optimizing compiler
- 1959. QR algorithm
- 1962. Quicksort
- **1965. Fast Fourier transform**
- 1977. Integer relation detection
- 1987. Fast multipole method

值得说明的是，离散 Fourier 逆变换 (IDFT) 的快速算法可由快速 Fourier 变换导出：

$$f = \text{IFFT}(\tilde{f}_{\text{DFT}}) = \frac{1}{N} \cdot \overline{\text{FFT}(\tilde{f}_{\text{DFT}})}$$

因此我们无需单独为离散 Fourier 逆变换开发快速算法。

4.3.1 一个直观的例子

考虑 $N = 4$ 的情形：

$$\begin{aligned} \tilde{f}_{\text{DFT}}[j] &= \sum_{k=0}^3 f[k] \exp\left(-\frac{i2\pi jk}{4}\right) = \sum_{k=0}^3 f[k] \omega^{jk} \\ \vec{\tilde{f}}_{\text{DFT}} &= \begin{bmatrix} \tilde{f}_{\text{DFT}}[0] \\ \tilde{f}_{\text{DFT}}[1] \\ \tilde{f}_{\text{DFT}}[2] \\ \tilde{f}_{\text{DFT}}[3] \end{bmatrix} = \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \omega^{0 \cdot 2} & \omega^{0 \cdot 3} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \omega^{1 \cdot 2} & \omega^{1 \cdot 3} \\ \omega^{2 \cdot 0} & \omega^{2 \cdot 1} & \omega^{2 \cdot 2} & \omega^{2 \cdot 3} \\ \omega^{3 \cdot 0} & \omega^{3 \cdot 1} & \omega^{3 \cdot 2} & \omega^{3 \cdot 3} \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ f[2] \\ f[3] \end{bmatrix} = V \cdot \vec{f} \\ \text{where } \omega &:= \exp\left(-\frac{i2\pi}{4}\right) \\ \text{and } V_{j,k} &= \exp\left(-\frac{i2\pi jk}{4}\right) = \omega^{jk} \end{aligned}$$

如果我们不考虑复指数部分的运算，

则求解离散 Fourier 变换共需 $4 \times 4 = 16$ 次复数乘法和 $4 \times (4 - 1) = 12$ 次复数加法。

注意到加法和乘法的分配律 $a \times b + a \times c = a \times (b + c)$ 是唯一可优化的地方，

于是我们有：

$$\begin{aligned} \tilde{f}_{\text{DFT}}[j] &= \sum_{k=0}^3 f[k] \exp\left(-\frac{i2\pi jk}{4}\right) \\ &= \sum_{k=0}^3 f[k] \omega^{jk} \\ &= f[0]\omega^{j \cdot 0} + f[1]\omega^{j \cdot 1} + f[2]\omega^{j \cdot 2} + f[3]\omega^{j \cdot 3} \\ &= (f[0] + f[2]\omega^{j \cdot 2}) \cdot \omega^{j \cdot 0} + (f[1] + f[3]\omega^{j \cdot 2}) \cdot \omega^{j \cdot 1} \\ \vec{\tilde{f}}_{\text{DFT}}[0] &= (f[0] + f[2] \cdot \omega^0) \cdot \omega^0 + (f[1] + f[3] \cdot \omega^0) \cdot \omega^0 \\ \vec{\tilde{f}}_{\text{DFT}}[1] &= (f[0] + f[2] \cdot \omega^2) \cdot \omega^0 + (f[1] + f[3] \cdot \omega^2) \cdot \omega^1 \\ \vec{\tilde{f}}_{\text{DFT}}[2] &= (f[0] + f[2] \cdot \omega^4) \cdot \omega^0 + (f[1] + f[3] \cdot \omega^4) \cdot \omega^2 \\ &= (f[0] + f[2] \cdot \omega^0) \cdot \omega^0 + (f[1] + f[3] \cdot \omega^0) \cdot \omega^2 \\ \vec{\tilde{f}}_{\text{DFT}}[3] &= (f[0] + f[2] \cdot \omega^6) \cdot \omega^0 + (f[1] + f[3] \cdot \omega^6) \cdot \omega^3 \\ &= (f[0] + f[2] \cdot \omega^2) \cdot \omega^0 + (f[1] + f[3] \cdot \omega^2) \cdot \omega^3 \end{aligned}$$

优化后求解离散 Fourier 变换只需 4 次复数乘法和 8 次复数加法：

$$\begin{aligned}
A &= f[0] + f[2] \\
B &= f[1] + f[3] \\
C &= f[1] + f[2] \cdot \omega^2 \\
D &= f[1] + f[3] \cdot \omega^2
\end{aligned}$$

$$\begin{aligned}
\tilde{f}_{\text{DFT}}[0] &= A + B \\
\tilde{f}_{\text{DFT}}[1] &= C + D \cdot \omega \\
\tilde{f}_{\text{DFT}}[2] &= A + B \cdot \omega^2 = A - B \\
\tilde{f}_{\text{DFT}}[3] &= C + D \cdot \omega^3 = C - D \cdot \omega \quad (\text{reuse } D \cdot \omega)
\end{aligned}$$

本质上, FFT 是一个矩阵分解算法:

$$\begin{aligned}
\tilde{f}_{\text{DFT}} &= \begin{bmatrix} \tilde{f}_{\text{DFT}}[0] \\ \tilde{f}_{\text{DFT}}[1] \\ \tilde{f}_{\text{DFT}}[2] \\ \tilde{f}_{\text{DFT}}[3] \end{bmatrix} = \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \omega^{0 \cdot 2} & \omega^{0 \cdot 3} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \omega^{1 \cdot 2} & \omega^{1 \cdot 3} \\ \omega^{2 \cdot 0} & \omega^{2 \cdot 1} & \omega^{2 \cdot 2} & \omega^{2 \cdot 3} \\ \omega^{3 \cdot 0} & \omega^{3 \cdot 1} & \omega^{3 \cdot 2} & \omega^{3 \cdot 3} \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ f[2] \\ f[3] \end{bmatrix} = V \cdot \vec{f} \\
&= \begin{bmatrix} \omega^0 & \omega^0 & & \\ \omega^0 & \omega^1 & & \\ & & \omega^0 & \omega^2 \\ & & \omega^0 & \omega^3 \end{bmatrix} \begin{bmatrix} \omega^0 & & \omega^0 & \\ & \omega^0 & & \omega^0 \\ \omega^0 & & \omega^2 & \\ & \omega^0 & & \omega^2 \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ f[2] \\ f[3] \end{bmatrix}
\end{aligned}$$

出于方便实际实现的某些原因 (我们在后面会介绍), 我们调换 $\tilde{f}_{\text{DFT}}[1]$ 和 $\tilde{f}_{\text{DFT}}[2]$ 的位置:

$$\begin{aligned}
\begin{bmatrix} \tilde{f}_{\text{DFT}}[0] \\ \tilde{f}_{\text{DFT}}[2] \\ \tilde{f}_{\text{DFT}}[1] \\ \tilde{f}_{\text{DFT}}[3] \end{bmatrix} &= \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \omega^{0 \cdot 2} & \omega^{0 \cdot 3} \\ \omega^{2 \cdot 0} & \omega^{2 \cdot 1} & \omega^{2 \cdot 2} & \omega^{2 \cdot 3} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \omega^{1 \cdot 2} & \omega^{1 \cdot 3} \\ \omega^{3 \cdot 0} & \omega^{3 \cdot 1} & \omega^{3 \cdot 2} & \omega^{3 \cdot 3} \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ f[2] \\ f[3] \end{bmatrix} \\
&= \begin{bmatrix} \omega^0 & \omega^0 & & \\ \omega^0 & \omega^2 & & \\ & & \omega^0 & \omega^1 \\ & & \omega^0 & \omega^3 \end{bmatrix} \begin{bmatrix} \omega^0 & & \omega^0 & \\ & \omega^0 & & \omega^0 \\ \omega^0 & & \omega^2 & \\ & \omega^0 & & \omega^2 \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ f[2] \\ f[3] \end{bmatrix}
\end{aligned}$$

更一般地, 对于 $N = 2^r$, FFT 算法将把原矩阵分解为 $r = \log_2(N)$ 个 $N \times N$ 矩阵的乘积, 每个因子矩阵具有最小数目的复数加法和复数乘法运算, 总共需要 $\frac{1}{2}Nr$ 次复数乘法和 Nr 次复数加法。

而直接法总共需要 N^2 次复数乘法和 $N(N-1)$ 次复数加法。

当 $N = 2^{10} = 1024$ 时, FFT 算法可将复数乘法的次数降为直接法的 $\frac{r}{N} = \frac{10}{1024} \approx 0.01$ 倍。

4.3.2 以 2 为底的 FFT 算法

通常来说, 离散 Fourier 变换的 N 是有选择的, 可取 $N = 2^r$

即使 N 不是 2 的幂, 我们也可以将其补成 2 的幂。

对于大的素数 (例如 $N = 65537$) 我们也有相应的 FFT 算法。

简单起见, 设 $N = 2^r$, 考虑一维离散 Fourier 变换:

$$\begin{aligned}
\tilde{f}_{\text{DFT}}[j] &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{\text{i}2\pi jk}{N}\right) \quad (j = 0, 1, \dots, N-1) \\
&\quad \Updownarrow \\
f[k] &= \frac{1}{N} \sum_{j=0}^{N-1} \tilde{f}_{\text{DFT}}[j] \cdot \exp\left(\frac{\text{i}2\pi jk}{N}\right) \quad (k = 0, 1, \dots, N-1)
\end{aligned}$$

我们将 j, k 记为二进制:

$$\begin{aligned}
j &= [j_{r-1}, j_{r-2}, \dots, j_0] = 2^{r-1} \cdot j_{r-1} + \dots + 2 \cdot j_1 + j_0 \\
k &= [k_{r-1}, k_{r-2}, \dots, k_0] = 2^{r-1} \cdot k_{r-1} + \dots + 2 \cdot k_1 + k_0
\end{aligned}$$

于是我们有:

$$\begin{aligned}
\tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_0]) &= \tilde{f}_{\text{DFT}}[j] \\
&= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{i2\pi jk}{N}\right) \\
&= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{jk} \\
&= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{2^{r-1}jk_{r-1}} \cdot \omega^{2^{r-2}jk_{r-2}} \cdots \omega^{2^0jk_0}
\end{aligned}$$

其中 $\omega = \exp(-i2\pi/N)$

注意到:

$$\begin{aligned}
\omega^{2^{r-1}jk_{r-1}} &= \omega^{2^{r-1}(j_0+2j_1+\cdots+2^{r-1}j_{r-1}) \cdot k_{r-1}} \\
&= \omega^{2^{r-1}j_0k_{r-1}} \cdot \omega^{2^rj_1k_{r-1}} \cdots \omega^{2^{2r-2}j_{r-1}k_{r-1}} \\
&\quad (\text{note that } \omega^{2^r \cdot l} = \omega^{N \cdot l} = 1^l = 1 \text{ for all integer } l) \\
&= \omega^{2^{r-1}j_0k_{r-1}} \cdot 1 \cdots 1 \\
&= \omega^{2^{r-1}j_0k_{r-1}} \\
\hline
\omega^{2^{r-2}jk_{r-2}} &= \omega^{2^{r-2}(j_0+2j_1+\cdots+2^{r-1}j_{r-1}) \cdot k_{r-2}} \\
&= \omega^{2^{r-2}j_0k_{r-2}} \cdot \omega^{2^{r-1}j_1k_{r-2}} \cdots \omega^{2^{2r-3}j_{r-1}k_{r-2}} \\
&\quad (\text{note that } \omega^{2^r \cdot l} = \omega^{N \cdot l} = 1^l = 1 \text{ for all integer } l) \\
&= \omega^{2^{r-2}j_0k_{r-2}} \cdot \omega^{2^{r-1}j_1k_{r-2}} \cdot 1 \cdots 1 \\
&= \omega^{2^{r-2}(j_0+2j_1)k_{r-2}} \\
\hline
\omega^{2^{r-p}jk_{r-p}} &= \omega^{2^{r-p}(j_0+2j_1+\cdots+2^{p-1}j_{p-1})k_{r-p}} \text{ for } p = 1, 2, \dots, r
\end{aligned}$$

因此我们有:

$$\begin{aligned}
\tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_0]) &= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{2^{r-1}jk_{r-1}} \cdot \omega^{2^{r-2}jk_{r-2}} \cdots \omega^{2^0jk_0} \\
&= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{2^{r-1}j_0k_{r-1}} \cdot \omega^{2^{r-2}(j_0+2j_1)k_{r-2}} \cdots \omega^{2^0(j_0+2j_1+\cdots+2^{r-1}j_{r-1})k_0}
\end{aligned}$$

利用分次求和, 并对中间结果标号, 即有:

$$\begin{aligned}
f_1([j_0, k_{r-2}, \dots, k_1, k_0]) &= \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega^{2^{r-1}j_0k_{r-1}} \\
&= f([0, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^{r-1}j_0 \cdot 0} + f([1, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^{r-1}j_0 \cdot 1} \\
&= f([0, k_{r-2}, \dots, k_1, k_0]) + f([1, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^{r-1}j_0} \\
\hline
f_2([j_0, j_1, k_{r-3}, \dots, k_1, k_0]) &= \sum_{k_{r-2}=0}^1 f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega^{2^{r-2}(j_0+2j_1)k_{r-2}} \\
&= f_1([j_0, 0, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^{r-2}(j_0+2j_1) \cdot 0} + f_1([j_0, 1, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^{r-2}(j_0+2j_1) \cdot 1} \\
&= f_1([j_0, 0, k_{r-3}, \dots, k_1, k_0]) + f_1([j_0, 1, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^{r-2}(j_0+2j_1)} \\
\hline
&\vdots \\
\hline
f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) &= \sum_{k_0=0}^1 f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega^{(j_0+2j_1+\cdots+2^{r-1}j_{r-1})k_0} \\
&= f_{r-1}([j_0, j_1, \dots, j_{r-2}, 0]) \omega^{(j_0+2j_1+\cdots+2^{r-1}j_{r-1}) \cdot 0} + f_{r-1}([j_0, j_1, \dots, j_{r-2}, 1]) \omega^{(j_0+2j_1+\cdots+2^{r-1}j_{r-1}) \cdot 1} \\
&= f_{r-1}([j_0, j_1, \dots, j_{r-2}, 0]) + f_{r-1}([j_0, j_1, \dots, j_{r-2}, 1]) \cdot \omega^{(j_0+2j_1+\cdots+2^{r-1}j_{r-1})} \\
&\text{finally } \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_1, j_0]) = f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}])
\end{aligned}$$

上述递推公式正是 Cooley 和 Tukey 提出的 FFT 算法的原始形式.

它共有 r 重和式, 每重和式需要 N 次复数乘法和 N 次复数加法,

因此总运算量为 Nr 次复数乘法和 Nr 次复数加法.

如果考虑到对称性 $\omega^{l+N/2} = -\omega^l$, 则复数乘法的运算次数还可减少一半.

Radix-2 FFT (assume $n = 2^m$)

- Idea: divide and conquer + distributive law

- $(E_0, E_1, \dots, E_{n/2-1}) = \text{FFT}(x_0, x_2, \dots, x_{n-2})$
- $(O_0, O_1, \dots, O_{n/2-1}) = \text{FFT}(x_1, x_3, \dots, x_{n-1})$
- for $k = 0, 1, \dots, n/2 - 1$

$$X_k = E_k + \omega_n^k O_k$$
- for $k = n/2, n/2 + 1, \dots, n - 1$

$$X_k = E_{k-n/2} + \omega_n^k O_{k-n/2}$$

- Complexity:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$$

with $O(1)$ workspace

Cooley-Tukey 算法 (Decimation in Time, DIT) 采用了分解指标 k 的办法.

若采用分解指标 j 的办法, 则导出的 FFT 算法称为 **Sande-Tukey 算法** (Decimation in Frequency, DIF)

$$\begin{aligned} \tilde{f}_{\text{DFT}}([j_{r-1}, \dots, j_1, j_0]) &= \tilde{f}_{\text{DFT}}[j] \\ &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{i2\pi jk}{N}\right) \\ &= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, \dots, k_1, k_0]) \omega^{jk} \\ &= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, \dots, k_1, k_0]) \omega^{2^{r-1}j_{r-1}k} \cdot \omega^{2^{r-2}j_{r-2}k} \cdots \omega^{2^0j_0k} \\ &= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \cdots \sum_{k_{r-1}=0}^1 f([k_{r-1}, \dots, k_1, k_0]) \omega^{2^{r-1}j_{r-1}k_0} \cdot \omega^{2^{r-2}j_{r-2}(k_0+2k_1)} \cdots \omega^{2^0j_0(k_0+2k_1+\cdots+2^{r-1}k_{r-1})} \end{aligned}$$

Sande-Tukey 算法的迭代公式如下:

$$\begin{aligned} f_1([j_0, k_{r-2}, \dots, k_1, k_0]) &= \sum_{k_{r-1}=0}^1 f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega^{2^0j_0(k_0+2k_1+\cdots+2^{r-2}k_{r-2}+2^{r-1}k_{r-1})} \\ &= f([0, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^0j_0(k_0+2k_1+\cdots+2^{r-2}k_{r-2}+2^{r-1} \cdot 0)} \\ &\quad + f([1, k_{r-2}, \dots, k_1, k_0]) \cdot \omega^{2^0j_0(k_0+2k_1+\cdots+2^{r-2}k_{r-2}+2^{r-1} \cdot 1)} \\ \hline f_2([j_0, j_1, k_{r-3}, \dots, k_1, k_0]) &= \sum_{k_{r-2}=0}^1 f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega^{2^1j_1(k_0+2k_1+\cdots+2^{r-3}k_{r-3}+2^{r-2}k_{r-2})} \\ &= f_1([j_0, 0, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^1j_1(k_0+2k_1+\cdots+2^{r-3}k_{r-3}+2^{r-2} \cdot 0)} \\ &\quad + f_1([j_0, 1, k_{r-3}, \dots, k_1, k_0]) \cdot \omega^{2^1j_1(k_0+2k_1+\cdots+2^{r-3}k_{r-3}+2^{r-2} \cdot 1)} \\ \hline &\vdots \\ \hline f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) &= \sum_{k_0=0}^1 f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega^{2^{r-1}j_{r-1}k_0} \\ &= f_{r-1}([j_0, j_1, \dots, j_{r-2}, 0]) \omega^{2^{r-1}j_{r-1} \cdot 0} + f_{r-1}([j_0, j_1, \dots, j_{r-2}, 1]) \omega^{2^{r-1}j_{r-1} \cdot 1} \\ &\text{finally } \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_1, j_0]) = f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) \end{aligned}$$

从程序设计的角度来讲, Sande-Tukey 算法具有更好的效率.

因为计算过程中 ω 的幂次是降序的, 而 Cooley-Tukey 算法则是跳跃的.

不过两者的运算量是一样的.

在上述讨论中，我们引入了中间变量 $f_1(k), f_2(k), \dots, f_r(k)$

但实际运算中我们不需要使用额外的内存保存这些中间变量。

例如 Cooley-Tukey 算法计算 $f_1([0, k_{r-1}, \dots, k_1, k_0])$ 和 $f_1([1, k_{r-1}, \dots, k_1, k_0])$ 时，只用到了 $f([0, k_{r-1}, \dots, k_1, k_0])$ 和 $f([1, k_{r-1}, \dots, k_1, k_0])$ ，

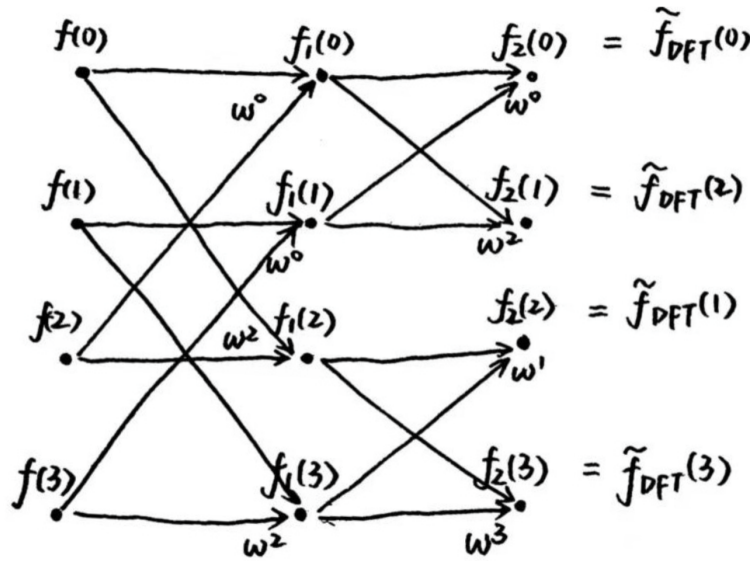
而其他中间变量的计算不会用到 $f([0, k_{r-1}, \dots, k_1, k_0])$ 和 $f([1, k_{r-1}, \dots, k_1, k_0])$ ，因此我们可用 $f_1([0, k_{r-1}, \dots, k_1, k_0])$ 的值覆盖 $f([0, k_{r-1}, \dots, k_1, k_0])$ 的存储单元，用 $f_1([1, k_{r-1}, \dots, k_1, k_0])$ 的值覆盖 $f([1, k_{r-1}, \dots, k_1, k_0])$ 的存储单元。

最终 $\tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_1, j_0])$ 的结果

将在第 $[j_0, j_1, \dots, j_{r-2}, j_{r-1}]$ 个存储单元 (即 $f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}])$) 中找到。

以 $N = 4$ 的情况的 Cooley-Tukey 算法为例：

$$\begin{aligned}
 f_1[0] &= f_1([0, 0]) = f([0, 0]) + f([1, 0]) \cdot \omega^0 \\
 f_1[2] &= f_1([1, 0]) = f([0, 0]) + f([1, 0]) \cdot \omega^2 \\
 f_1[1] &= f_1([0, 1]) = f([0, 1]) + f([1, 1]) \cdot \omega^0 \\
 f_1[3] &= f_1([1, 1]) = f([0, 1]) + f([1, 1]) \cdot \omega^2 \\
 \hline
 f_2[0] &= f_2([0, 0]) = f_1([0, 0]) + f_1([0, 1]) \cdot \omega^0 \\
 f_2[1] &= f_2([0, 1]) = f_1([0, 0]) + f_1([0, 1]) \cdot \omega^2 \\
 f_2[2] &= f_2([1, 0]) = f_1([1, 0]) + f_1([1, 1]) \cdot \omega^1 \\
 f_2[3] &= f_2([1, 1]) = f_1([1, 0]) + f_1([1, 1]) \cdot \omega^3 \\
 \hline
 \tilde{f}_{\text{DFT}}[0] &= \tilde{f}_{\text{DFT}}([0, 0]) = f_2([0, 0]) = f_2[0] \\
 \tilde{f}_{\text{DFT}}[1] &= \tilde{f}_{\text{DFT}}([0, 1]) = f_2([1, 0]) = f_2[2] \\
 \tilde{f}_{\text{DFT}}[2] &= \tilde{f}_{\text{DFT}}([1, 0]) = f_2([0, 1]) = f_2[1] \\
 \tilde{f}_{\text{DFT}}[3] &= \tilde{f}_{\text{DFT}}([1, 1]) = f_2([1, 1]) = f_2[3]
 \end{aligned}$$

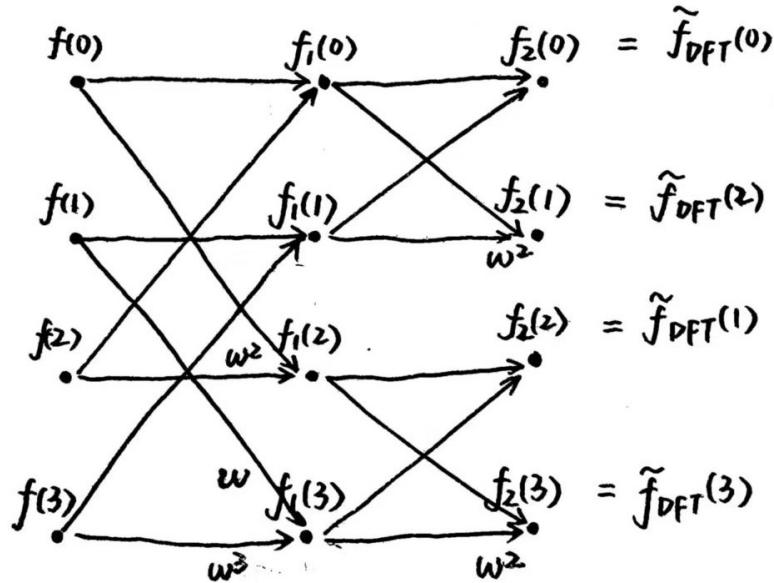


这也是为什么我们要调换 $\tilde{f}_{\text{DFT}}[1]$ 和 $\tilde{f}_{\text{DFT}}[2]$ 的位置：

$$\begin{aligned}
 \begin{bmatrix} \tilde{f}_{\text{DFT}}[0] \\ \tilde{f}_{\text{DFT}}[2] \\ \tilde{f}_{\text{DFT}}[1] \\ \tilde{f}_{\text{DFT}}[3] \end{bmatrix} &= \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \omega^{0 \cdot 2} & \omega^{0 \cdot 3} \\ \omega^{2 \cdot 0} & \omega^{2 \cdot 1} & \omega^{2 \cdot 2} & \omega^{2 \cdot 3} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \omega^{1 \cdot 2} & \omega^{1 \cdot 3} \\ \omega^{3 \cdot 0} & \omega^{3 \cdot 1} & \omega^{3 \cdot 2} & \omega^{3 \cdot 3} \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ f[2] \\ f[3] \end{bmatrix} \\
 &= \begin{bmatrix} \omega^0 & \omega^0 & & \\ \omega^0 & \omega^2 & & \\ & & \omega^0 & \omega^1 \\ & & \omega^0 & \omega^3 \end{bmatrix} \begin{bmatrix} \omega^0 & & \omega^0 & \\ & \omega^0 & & \omega^0 \\ \omega^0 & & \omega^2 & \\ & \omega^0 & & \omega^2 \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ f[2] \\ f[3] \end{bmatrix}
 \end{aligned}$$

以 $N = 4$ 的情况的 Sande-Tukey 算法为例：

$$\begin{aligned}
f_1[0] &= f_1([0, 0]) = f([0, 0]) \cdot \omega^0 + f([1, 0]) \cdot \omega^0 \\
f_1[2] &= f_1([1, 0]) = f([0, 0]) \cdot \omega^0 + f([1, 0]) \cdot \omega^2 \\
f_1[1] &= f_1([0, 1]) = f([0, 1]) \cdot \omega^0 + f([1, 1]) \cdot \omega^0 \\
f_1[3] &= f_1([1, 1]) = f([0, 1]) \cdot \omega^1 + f([1, 1]) \cdot \omega^3 \\
\hline
f_2[0] &= f_2([0, 0]) = f_1([0, 0]) \cdot \omega^0 + f_1([0, 1]) \cdot \omega^0 \\
f_2[1] &= f_2([0, 1]) = f_1([0, 0]) \cdot \omega^0 + f_1([0, 1]) \cdot \omega^2 \\
f_2[2] &= f_2([1, 0]) = f_1([1, 0]) \cdot \omega^0 + f_1([1, 1]) \cdot \omega^0 \\
f_2[3] &= f_2([1, 1]) = f_1([1, 0]) \cdot \omega^0 + f_1([1, 1]) \cdot \omega^2 \\
\hline
\tilde{f}_{\text{DFT}}[0] &= \tilde{f}_{\text{DFT}}([0, 0]) = f_2([0, 0]) = f_2[0] \\
\tilde{f}_{\text{DFT}}[1] &= \tilde{f}_{\text{DFT}}([0, 1]) = f_2([1, 0]) = f_2[2] \\
\tilde{f}_{\text{DFT}}[2] &= \tilde{f}_{\text{DFT}}([1, 0]) = f_2([0, 1]) = f_2[1] \\
\tilde{f}_{\text{DFT}}[3] &= \tilde{f}_{\text{DFT}}([1, 1]) = f_2([1, 1]) = f_2[3]
\end{aligned}$$



其矩阵形式如下:

$$\begin{aligned}
\begin{bmatrix} \tilde{f}_{\text{DFT}}[0] \\ \tilde{f}_{\text{DFT}}[2] \\ \tilde{f}_{\text{DFT}}[1] \\ \tilde{f}_{\text{DFT}}[3] \end{bmatrix} &= \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \omega^{0 \cdot 2} & \omega^{0 \cdot 3} \\ \omega^{2 \cdot 0} & \omega^{2 \cdot 1} & \omega^{2 \cdot 2} & \omega^{2 \cdot 3} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \omega^{1 \cdot 2} & \omega^{1 \cdot 3} \\ \omega^{3 \cdot 0} & \omega^{3 \cdot 1} & \omega^{3 \cdot 2} & \omega^{3 \cdot 3} \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ f[2] \\ f[3] \end{bmatrix} \\
&= \begin{bmatrix} \omega^0 & \omega^0 & & \\ \omega^0 & \omega^2 & & \\ & & \omega^0 & \omega^0 \\ & & \omega^0 & \omega^2 \end{bmatrix} \begin{bmatrix} \omega^0 & & \omega^0 \\ & \omega^0 & & \omega^0 \\ \omega^0 & & \omega^2 & \\ & \omega^1 & & \omega^3 \end{bmatrix} \begin{bmatrix} f[0] \\ f[1] \\ f[2] \\ f[3] \end{bmatrix}
\end{aligned}$$

从流程图可以看出, 每一前排节点仅用于计算两个后续节点而对其他节点毫无用处.

4.3.3 以 s 为底的 FFT 算法

设 $N = s^r$, 考虑一维离散 Fourier 变换:

$$\begin{aligned}
\tilde{f}_{\text{DFT}}[j] &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{i2\pi jk}{N}\right) \quad (j = 0, 1, \dots, N-1) \\
&\quad \Updownarrow \\
f[k] &= \frac{1}{N} \sum_{j=0}^{N-1} \tilde{f}_{\text{DFT}}[j] \cdot \exp\left(\frac{i2\pi jk}{N}\right) \quad (k = 0, 1, \dots, N-1)
\end{aligned}$$

我们将 j, k 记为 p 进制:

$$j = [j_{r-1}, j_{r-2}, \dots, j_0] = s^{r-1} \cdot j_{r-1} + \dots + s \cdot j_1 + j_0$$

$$k = [k_{r-1}, k_{r-2}, \dots, k_0] = s^{r-1} \cdot k_{r-1} + \dots + s \cdot k_1 + k_0$$

于是我们有:

$$\begin{aligned} \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_0]) &= \tilde{f}_{\text{DFT}}[j] \\ &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{i2\pi jk}{N}\right) \\ &= \sum_{k_0=0}^{s-1} \sum_{k_1=0}^{s-1} \dots \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{jk} \\ &= \sum_{k_0=0}^{s-1} \sum_{k_1=0}^{s-1} \dots \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{s^{r-1}jk_{r-1}} \cdot \omega^{s^{r-2}jk_{r-2}} \dots \omega^{s^0jk_0} \end{aligned}$$

其中 $\omega = \exp(-i2\pi/N)$

注意到:

$$\begin{aligned} \omega^{s^{r-1}jk_{r-1}} &= \omega^{s^{r-1}(j_0+s \cdot j_1+\dots+s^{r-1} \cdot j_{r-1}) \cdot k_{r-1}} \\ &= \omega^{s^{r-1}j_0k_{r-1}} \cdot \omega^{s^r j_1 k_{r-1}} \dots \omega^{s^{2r-2} j_{r-1} k_{r-1}} \\ &\quad (\text{note that } \omega^{s^r \cdot l} = \omega^{N \cdot l} = 1^l = 1 \text{ for all integer } l) \\ &= \omega^{s^{r-1}j_0k_{r-1}} \cdot 1 \dots 1 \\ &= \omega^{s^{r-1}j_0k_{r-1}} \end{aligned}$$

$$\begin{aligned} \omega^{s^{r-2}jk_{r-2}} &= \omega^{s^{r-2}(j_0+s \cdot j_1+\dots+s^{r-1} \cdot j_{r-1}) \cdot k_{r-2}} \\ &= \omega^{s^{r-2}j_0k_{r-2}} \cdot \omega^{s^{r-1}j_1k_{r-2}} \dots \omega^{s^{2r-3} j_{r-1} k_{r-2}} \\ &\quad (\text{note that } \omega^{s^r \cdot l} = \omega^{N \cdot l} = 1^l = 1 \text{ for all integer } l) \\ &= \omega^{s^{r-2}j_0k_{r-2}} \cdot \omega^{s^{r-1}j_1k_{r-2}} \cdot 1 \dots 1 \\ &= \omega^{s^{r-2}(j_0+s \cdot j_1)k_{r-2}} \end{aligned}$$

$$\omega^{s^{r-p}jk_{r-p}} = \omega^{s^{r-p}(j_0+s \cdot j_1+\dots+s^{p-1} \cdot j_{p-1})k_{r-p}} \text{ for } p = 1, 2, \dots, r$$

因此我们有:

$$\begin{aligned} \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_0]) &= \sum_{k_0=0}^{s-1} \sum_{k_1=0}^{s-1} \dots \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{s^{r-1}jk_{r-1}} \cdot \omega^{s^{r-2}jk_{r-2}} \dots \omega^{s^0jk_0} \\ &= \sum_{k_0=0}^{s-1} \sum_{k_1=0}^{s-1} \dots \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_0]) \omega^{s^{r-1}j_0k_{r-1}} \cdot \omega^{s^{r-2}(j_0+s \cdot j_1)k_{r-2}} \dots \omega^{s^0(j_0+s \cdot j_1+\dots+s^{r-1}j_{r-1})k_0} \end{aligned}$$

利用分次求和, 并对中间结果标号, 即有:

$$\begin{aligned} f_1([j_0, k_{r-2}, \dots, k_1, k_0]) &= \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega^{s^{r-1}j_0k_{r-1}} \\ f_2([j_0, j_1, k_{r-3}, \dots, k_1, k_0]) &= \sum_{k_{r-2}=0}^{s-1} f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega^{s^{r-2}(j_0+s \cdot j_1)k_{r-2}} \\ &\quad \dots \\ f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) &= \sum_{k_0=0}^{s-1} f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega^{(j_0+s \cdot j_1+\dots+s^{r-1}j_{r-1})k_0} \\ \text{finally } \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_1, j_0]) &= f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) \end{aligned}$$

上述递推公式正是 Cooley 和 Tukey 提出的 FFT 算法的原始形式.

它共有 r 重和式, 每重和式需要 $(s-1)N$ 次复数乘法 and $(s-1)N$ 次复数加法,

因此总运算量为 $(s-1)Nr$ 次复数乘法 and $(s-1)Nr$ 次复数加法,

这相对于以 2 为底的 FFT 算法似乎没有多少优势.

有趣的是, 对上式重新分组可得如下形式:

Denote $\omega_s := \omega^{s^{r-1}}$

$$\begin{aligned}
 f_1([j_0, k_{r-2}, \dots, k_1, k_0]) &= \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega^{s^{r-1} j_0 k_{r-1}} \\
 &= \left[\sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega_s^{j_0 k_{r-1}} \right] \\
 f_2([j_0, j_1, k_{r-3}, \dots, k_1, k_0]) &= \sum_{k_{r-2}=0}^{s-1} f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega^{s^{r-2}(j_0 + s j_1) k_{r-2}} \\
 &= \left[\sum_{k_{r-2}=0}^{s-1} f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega_s^{j_1 k_{r-2}} \right] \omega^{s^{r-2} j_0 k_{r-2}} \\
 &\dots \\
 f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) &= \sum_{k_0=0}^{s-1} f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega^{(j_0 + s j_1 + \dots + s^{r-1} j_{r-1}) k_0} \\
 &= \left[\sum_{k_0=0}^{s-1} f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega_s^{j_{r-1} k_0} \right] \omega^{(j_0 + s j_1 + \dots + s^{r-2} j_{r-2}) k_0} \\
 &\text{finally } \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_1, j_0]) = f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}])
 \end{aligned}$$

上述为 **Cooley-Tukey 形式的旋转因子法**.

注意到上式的括号部分恰好是 s 点的离散 Fourier 变换.

当 $s = 4$ 时, $\omega_4^k = \pm 1$ or $\pm i$, 因此上式的括号部分无须乘法运算.

此时它只需 Nr 个复数乘法法和 $3Nr$ 个复数加法,

而对于 $N = 4^r = 2^{2r}$ 用以 2 为底的算法则需要 $2Nr$ 个复数乘法法和 $2Nr$ 个复数加法.

- Radix- p FFT (assume $n = p^m$)

- Divide: p FFTs of length n/p
- Combine: $\Theta(p)$ work for each X_k
- Complexity:

$$T(n) = pT\left(\frac{n}{p}\right) + \Theta(pn) \Rightarrow T(n) = \Theta(n \log n)$$

- General FFT (assume $n = p_1 p_2 \dots p_m$)

- Divide: p_m FFTs of length n/p_m
- Combine: $\Theta(p_m)$ work for each X_k

Additional effort is required if n has a large prime factor

Cooley-Tukey 算法 (Decimation in Time, DIT) 采用了分解指标 k 的办法.

若采用分解指标 j 的办法, 则导出的 FFT 算法称为 **Sande-Tukey 算法** (Decimation in Frequency, DIF)

$$\begin{aligned}
 \tilde{f}_{\text{DFT}}([j_{r-1}, \dots, j_1, j_0]) &= \tilde{f}_{\text{DFT}}[j] \\
 &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{i2\pi jk}{N}\right) \\
 &= \sum_{k_0=0}^{s-1} \sum_{k_1=0}^{s-1} \dots \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, \dots, k_1, k_0]) \omega^{jk} \\
 &= \sum_{k_0=0}^{s-1} \sum_{k_1=0}^{s-1} \dots \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, \dots, k_1, k_0]) \omega^{s^{r-1} j_{r-1} k} \cdot \omega^{s^{r-2} j_{r-2} k} \dots \omega^{s^0 j_0 k} \\
 &= \sum_{k_0=0}^{s-1} \sum_{k_1=0}^{s-1} \dots \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, \dots, k_1, k_0]) \omega^{s^{r-1} j_{r-1} k_0} \cdot \omega^{s^{r-2} j_{r-2} (k_0 + s k_1)} \dots \omega^{s^0 j_0 (k_0 + s k_1 + \dots + s^{r-1} k_{r-1})}
 \end{aligned}$$

Sande-Tukey 算法的迭代公式如下:

$$\begin{aligned}
f_1([j_0, k_{r-2}, \dots, k_1, k_0]) &= \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega^{s^0 j_0 (k_0 + s k_1 + \dots + s^{r-2} k_{r-2} + s^{r-1} k_{r-1})} \\
f_2([j_0, j_1, k_{r-3}, \dots, k_1, k_0]) &= \sum_{k_{r-2}=0}^{s-1} f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega^{s^1 j_1 (k_0 + s k_1 + \dots + s^{r-3} k_{r-3} + s^{r-2} k_{r-2})} \\
&\dots \\
f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) &= \sum_{k_0=0}^{s-1} f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega^{s^{r-1} j_{r-1} k_0} \\
\text{finally } \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_1, j_0]) &= f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}])
\end{aligned}$$

从程序设计的角度来讲，Sande-Tukey 算法具有更好的效率。

因为计算过程中 ω 的幂次是降序的，而 Cooley-Tukey 算法则是跳跃的。

不过两者的运算量是一样的。

对上式重新分组可得 **以 s 为底的 Sande-Tukey 算法的旋转因子法**：

$$\begin{aligned}
&\text{Denote } \omega_s := \omega^{s^{r-1}} \\
f_1([j_0, k_{r-2}, \dots, k_1, k_0]) &= \sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega^{s^0 j_0 (k_0 + s k_1 + \dots + s^{r-2} k_{r-2} + s^{r-1} k_{r-1})} \\
&= \left[\sum_{k_{r-1}=0}^{s-1} f([k_{r-1}, k_{r-2}, \dots, k_1, k_0]) \omega_s^{j_0 k_{r-1}} \right] \omega^{j_0 (k_0 + s k_1 + \dots + s^{r-2} k_{r-2})} \\
f_2([j_0, j_1, k_{r-3}, \dots, k_1, k_0]) &= \sum_{k_{r-2}=0}^{s-1} f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega^{s^1 j_1 (k_0 + s k_1 + \dots + s^{r-3} k_{r-3} + s^{r-2} k_{r-2})} \\
&= \left[\sum_{k_{r-2}=0}^{s-1} f_1([j_0, k_{r-2}, k_{r-3}, \dots, k_1, k_0]) \omega_s^{j_1 k_{r-2}} \right] \omega^{s^1 j_1 (k_0 + s k_1 + \dots + s^{r-3} k_{r-3})} \\
&\dots \\
f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}]) &= \sum_{k_0=0}^{s-1} f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega^{s^{r-1} j_{r-1} k_0} \\
&= \left[\sum_{k_0=0}^{s-1} f_{r-1}([j_0, j_1, \dots, j_{r-2}, k_0]) \omega_s^{j_{r-1} k_0} \right] \\
\text{finally } \tilde{f}_{\text{DFT}}([j_{r-1}, j_{r-2}, \dots, j_1, j_0]) &= f_r([j_0, j_1, \dots, j_{r-2}, j_{r-1}])
\end{aligned}$$

Sande-Tukey 算法的旋转因子法和 Cooley-Tukey 算法的旋转因子法的括号部分是完全一样的，所不同的是尾部因子的表现形式。

从程序设计的角度讲，Sande-Tukey 算法的旋转因子法具有更好的效率，因为它可以减少 ω^p 的计算。

4.4 应用

4.4.1 卷积

设 $\{f[k]\}$ 和 $\{g[k]\}$ 是以 N 为周期的点列，则其 (循环) 离散卷积定义为：

$$(f \star g)[k] := \sum_{j=0}^{N-1} f[j] \cdot g[k-j] \quad (k = 0, 1, \dots, N-1)$$

直接使用上式进行计算，共需 N^2 次乘法和 $N(N-1)$ 次加法。

根据卷积定理 $(\widetilde{f \star g})_{\text{DFT}}[j] = \tilde{f}_{\text{DFT}}[j] \cdot \tilde{g}_{\text{DFT}}[j]$ ($j = 0, 1, \dots, N-1$) 可知，

我们可以将离散卷积分解为以下三步来做：

- ① 计算 $\{f[k]\}$ 和 $\{g[k]\}$ 的离散 Fourier 变换 (必须使用 FFT)

$$\begin{aligned}
\tilde{f}_{\text{DFT}}[j] &= \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{\text{i}2\pi jk}{N}\right) \\
\tilde{g}_{\text{DFT}}[j] &= \sum_{k=0}^{N-1} g[k] \exp\left(-\frac{\text{i}2\pi jk}{N}\right) \\
&\text{where } j = 0, 1, \dots, N-1
\end{aligned}$$

- ② 计算离散 Fourier 变换的乘积:

$$(\widetilde{f \star g})_{\text{DFT}}[j] = \tilde{f}_{\text{DFT}}[j] \cdot \tilde{g}_{\text{DFT}}[j] \quad (j = 0, 1, \dots, N-1)$$

- ③ 计算离散 Fourier 逆变换: (必须使用 IFFT)

$$(f \star g)[k] = \frac{1}{N} \sum_{j=0}^{N-1} (\widetilde{f \star g})_{\text{DFT}}[j] \exp\left(\frac{\mathrm{i}2\pi jk}{N}\right)$$

where $k = 0, 1, \dots, N-1$

其中离散 Fourier 逆变换 (IDFT) 的快速算法可由快速 Fourier 变换导出:

$$f = \text{IFFT}(\tilde{f}_{\text{DFT}}) = \frac{1}{N} \cdot \overline{\text{FFT}\left(\overline{\tilde{f}_{\text{DFT}}}\right)}$$

- Convolution:

$$(u * v)(s) = \int_{-\infty}^{+\infty} u(t)v(s-t) dt = (v * u)(s)$$

- Convolution theorem:

$$\widehat{u * v} = \widehat{u} \cdot \widehat{v}, \quad \widehat{u \cdot v} = \frac{1}{2\pi} \widehat{u} * \widehat{v},$$

where

$$\widehat{u} = \int_{-\infty}^{+\infty} u(t)e^{-ist} ds$$

The scaling factor in Fourier transform does matter!

- Parseval's identity: $\|\widehat{u}\|_2 = \sqrt{2\pi} \|u\|_2$
- Application: Make a function smoother

- Circular discrete convolution (for periodic sequences):

$$(u * v)_k = (v * u)_k = \sum_{j=0}^{n-1} u_j v_{k-j}$$

- Circular convolution theorem:

$$(\widehat{u * v})_k = \widehat{u}_k \cdot \widehat{v}_k, \quad (\widehat{u \cdot v})_k = \frac{1}{n} (\widehat{u} * \widehat{v})_k,$$

where

$$\widehat{u}_k = \sum_{j=0}^{n-1} \exp\left(-\frac{2\pi i}{n} \cdot jk\right) u_j$$

Again, the scaling factor does matter!

- Parseval's identity: $\|\widehat{u}\|_2 = \sqrt{n} \|u\|_2$

- General discrete convolution:

$$(u * v)_k = (v * u)_k = \sum_{0 \leq j \leq k} u_j v_{k-j},$$

where u and v can have different dimensions

$\rightsquigarrow$ reduces to circular convolution by padding zeros

- Naive convolution algorithm: $\Theta(n^2)$
- Fast convolution algorithms: $\Theta(n \log n)$ using FFT
- Applications:
 - Multiplication of big integers
 - Multiplication of polynomials

4.4.2 音乐

你甚至可以在数值算法课上学到音乐知识:

- ① 正弦波可合成音乐, 噪声波可作为节奏, 用 Octave 的 `sound` 指令播放
- ② **基音**是一个声音中频率最低的成分, 决定了我们感知到的音高.
标准音 $A_4 := 440 \text{ Hz}$
十二平均律是将一个八度 (即频率比为 $2:1$ 的音程) 平均分成 12 个半音的音律系统.
任意两个相邻音符之间的频率比为 $2^{1/12} \approx 1.059$
男高音达到 $C_5 \approx 523 \text{ Hz}$ 、女高音达到 $F_6 \approx 1397 \text{ Hz}$ 就非常厉害了.
([在线测音高](#)) 音乐频率的图像应该采用 $\log$ 尺度, 而不应采用自然尺度.
- ③ 当一个振动系统振动时, 除了整体振动 (基音) 外, 还会同时发生部分振动, 形成多个频率成分, 即**泛音**.
这些频率成分按照一定的比例排列, 形成**泛音列**:
基音 (第 1 泛音) f , 第 2 泛音 $2f$, 第 3 泛音 $3f \dots$

基音 + 泛音决定音色.
如果两个音的频率呈小整数比 (例如 A_4 和 D_4 呈 $3:2$),
那么它们的泛音列会有更多重合, 我们的大脑解码快,
听起来就会 "和谐", 否则听起来就会 "不和谐" (例如 A_4 和 G_4 呈 $11:10$)

双音多频 (Dual-Tone Multi-Frequency, DTMF)

通过同时发送两个不同频率的音调信号来代表数字、字母和符号,
从而实现用户与电话交换系统之间的交互:

Dual-Tone Multi-Frequency (DTMF) signal

- Keyboard

	1209 Hz	1336 Hz	1477 Hz	1633 Hz
697 Hz	1	2	3	A
770 Hz	4	5	6	B
852 Hz	7	8	9	C
941 Hz	*	0	#	D

- Each key produces a sum of two sine waves
 $\rightsquigarrow$ FFT can be used for decoding
- Practical approach: Goertzel's Algorithm

(Homework 11 Problem 5)

其解码可以使用 FFT 算法,
根据幅值大小解析出两个主要频率, 再查表得到对应的数字、字母和符号.
一种更高效的方法是 **Goertzel 算法**, 它可以用于识别特定频率成分,
对于 DTMF 我们只需识别 697, 770, 852, 941, 1209, 1336, 1477, 1633 Hz 的频率成分,
根据其幅值大小挑选出两个主要频率, 再查表得到对应的数字、字母和符号.

4.4.3 水印

([阿里巴巴内网不可见水印事件](#))

我们可以对图片进行离散 Fourier 变换, 加入 "水印" 的频域信号, 再进行离散 Fourier 逆变换.
最终得到的图片携带的水印是难以用肉眼识别的.

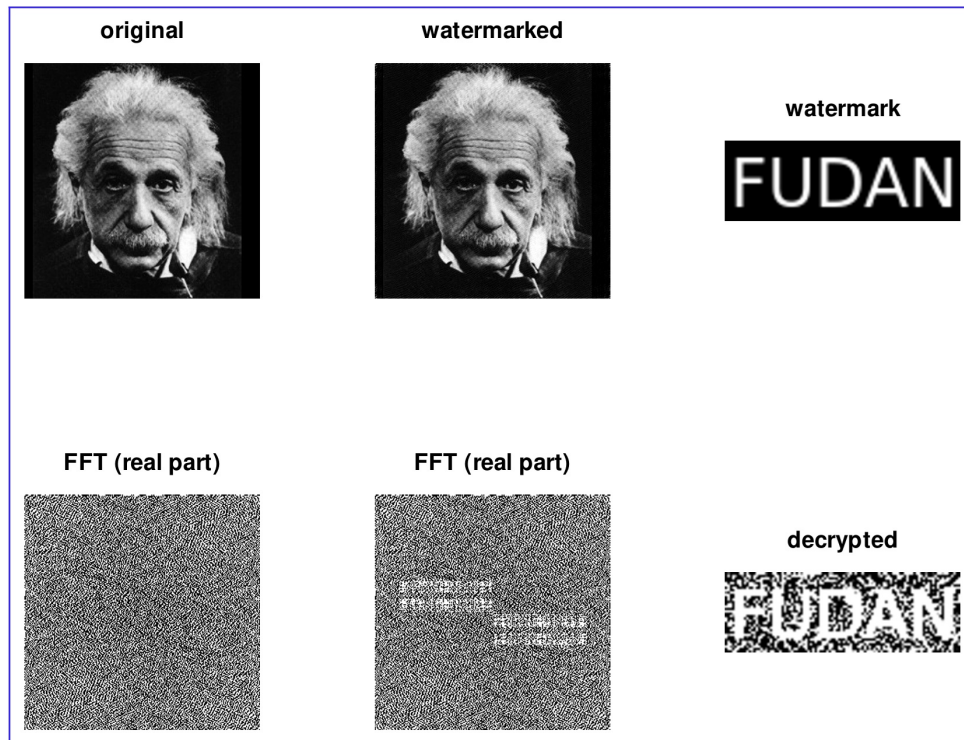
Invisible watermark

- Encoding

1. Transform a picture into frequency domain
2. Insert watermark in frequency domain
3. Transform back to real domain

- Decoding

1. Transform the encrypted picture into frequency domain
2. Extract watermark



4.4.4 微分方程

Fourier 变换 $\mathcal{F}[\cdot]$ 可以对角化求导算子:

$$\begin{aligned}\mathcal{F}[u](t) &= \int_{-\infty}^{\infty} u(s) \exp(-i2\pi st) ds \\ \mathcal{F}[u'](t) &= \int_{-\infty}^{\infty} u'(s) \exp(-i2\pi st) ds \\ &= u(s) \exp(-i2\pi st) \Big|_{-\infty}^{\infty} - \int_{-\infty}^{\infty} u(s) \cdot (-i2\pi t) \cdot \exp(-i2\pi st) ds \\ &= i2\pi t \int_{-\infty}^{\infty} u(s) \exp(-i2\pi st) ds \\ &= i2\pi t \cdot \mathcal{F}[u](t) \\ \mathcal{F}[u''](t) &= \int_{-\infty}^{\infty} u''(s) \exp(-i2\pi st) ds \\ &= u'(s) \exp(-i2\pi st) \Big|_{-\infty}^{\infty} - \int_{-\infty}^{\infty} u'(s) \cdot (-i2\pi t) \cdot \exp(-i2\pi st) ds \\ &= i2\pi t \int_{-\infty}^{\infty} u'(s) \exp(-i2\pi st) ds \\ &= i2\pi t \cdot \mathcal{F}[u'](t) \\ &= (i2\pi t)^2 \cdot \mathcal{F}[u](t)\end{aligned}$$

记 $v(t) := \mathcal{F}[u](t)$ 可知 $u(t) = \mathcal{F}^{-1}[v](t)$
 记求导算子为 $\mathcal{D}$, 则我们有:

$$\begin{aligned}\mathcal{F}\mathcal{D}\mathcal{F}^{-1}v(t) &= \mathcal{F}\mathcal{D}u(t) \\ &= \mathcal{F}u'(t) \\ &= i2\pi t \cdot \mathcal{F}u(t) \\ &= i2\pi t \cdot v(t)\end{aligned}$$

这说明在 Fourier 变换下求导算子 $\mathcal{D}$ 被对角化了.
 因此微分方程可将导数运算变为代数运算:

$$\begin{aligned}au''(t) + bu'(t) + cu(t) &= d(t) \\ \Updownarrow \\ (a \cdot (i2\pi t)^2 + b \cdot (i2\pi t) + c)\mathcal{F}[u](t) &= \mathcal{F}[d](t)\end{aligned}$$

这套变换更常使用 Laplace 变换, 但其逆变换要比 Fourier 逆变换更加复杂.

Fast Poisson solver

- Poisson's equation with Dirichlet boundary condition

$$\begin{cases} -\Delta u(x, y) = f(x, y), & \text{in } \Omega \\ u(x, y) = 0, & \text{on } \partial\Omega \end{cases}$$

- Finite difference discretization for rectangular domain

$$\begin{aligned}-\frac{\partial^2 u}{\partial x^2} &= \frac{-u_{i-1,j} + 2u_{i,j} - u_{i+1,j}}{h_x^2} + O(h_x^2) \\ -\frac{\partial^2 u}{\partial y^2} &= \frac{-u_{i,j-1} + 2u_{i,j} - u_{i,j+1}}{h_y^2} + O(h_y^2)\end{aligned}$$

- Matrix form: $T_x U + U T_y = F$
 $\rightsquigarrow$ can be solved by 2-D FFT

The End